Principle and Practice of Bidirectional Programming in BiGUL

Zhenjiang Hu, Hsiang-Shang Ko

National Institute of Informatics, Japan
{hu,hsiang-shang}@nii.ac.jp

Abstract. Putback-based bidirectional programming allows the programmer to write only one putback transformation, from which the unique corresponding forward transformation is derived for free. A key distinguishing feature of putback-based bidirectional programming is full control over the bidirectional behavior, which is important for specifying intended bidirectional transformations without any ambiguity. In this tutorial, we will introduce BiGUL, a simple yet powerful putback-based bidirectional programming language, explaining the underlying principles and showing how various kinds of bidirectional applications can be developed in BiGUL.

1 Putback-based bidirectional programming

In this tutorial, the kind of bidirectional transformations (BXs) we discuss is *aymmetric lenses* [7], which basically consist of a pair of transformations: a *forward* transformation *get* producing a *view* from a *source*, and a *backward*, or *putback*, transformation *put* which takes a source and a possibly modified view, and reflects the modifications on the view to the source, producing an updated source. These two transformations should be *well-behaved* in the sense that they satisfy the following round-tripping laws:

$$put\ s\ (get\ s) = s \qquad \textsc{GetPut}$$
$$get\ (put\ s\ v) = v \qquad \textsc{PutGet}$$

The GetPut property requires that no changing on the view should be reflected as no changing on the source, while the PutGet property requires that all changes in the view should be completely reflected to the source so that the changed view can be successfully recovered by applying the forward transformation to the updated source.

The purpose of *bidirectional programming* is to develop well-behaved bidirectional transformations to solve various synchronization problems. A straightforward approach to bidirectional programming is to write two unidirectional transformations. Although this ad hoc solution provides full control over both get and putback transformations, and can be realized using standard programming languages, the programmer needs to show that the two transformations

satisfy the well-behavedness laws, and a modification to one of the transformations requires a redefinition of the other transformation as well as a new well-behavedness proof. To ease and enable maintainable bidirectional programming, it is preferable to write just a single program that can denote both transformations.

Lots of work [1, 2, 7–9, 11, 14, 15] has been devoted to the *get-based* approach, allowing the programmer to write the forward transformation *get* and deriving a suitable putback transformation. While the get-based approach is friendly, a *get* function may not be injective, so there may exist many possible *put* functions that can be combined with it to form a valid BX, and there is no way to control the choice of *put* through the definition of *get*. This ambiguity of *put* is what makes bidirectional programming challenging and unpredictable in practice.

The main topic of this tutorial is the *putback-based* approach to bidirectional programming. In contrast to the get-based approach, it allows the programmer to write a backward transformation *put* and derives a suitable *get* that can be paired with this *put* to form a bidirectional transformation. Interestingly, while *get* usually loses information when mapping from a source to a view, *put* must preserve information when putting back from the view to the source, according to the PutGet property.

Before explaining how to program *put* in practice, let us briefly review the foundation [4–6], showing that "putback" is the essence of bidirectional programming. We start by defining validity of *put* as follows:

Definition 1 (Validity of *put*). *We say that a put is* valid *if there exists a get such that both* GetPut *and* PutGet *are satisfied.*

The first interesting fact is that, for a valid *put*, there exists exactly one *get* that can form a BX with it. This is in sharp contrast to get-based bidirectional programming, where many *put*s may be paired with a *get* to form a BX.

Lemma 1 (Uniqueness of *get*). *Given a put function, there exists at most one get function that forms a well-behaved BX.*

The second interesting fact is that it is possible to check validity of *put* without mentioning *get*. The following are two important properties on *put*.

- The first, which we call *view determination*, says that equivalence of updated sources produced by a *put* implies equivalence of views that are put back.

$$\forall\ s, s', v, v'.\ put\ s\ v\ =\ put\ s'\ v'\ \Rightarrow\ v\ =\ v' \qquad \textsc{ViewDetermination}$$

  Note that view determination implies that $put\ s$ is injective (with $s = s'$).
- The second, which we call *source stability*, denotes a slightly stronger notion of surjectivity for every source:

$$\forall\ s.\ \exists\ v.\ put\ s\ v\ =\ s \qquad \textsc{SourceStability}$$

Actually, these two properties together provide an equivalent characterization of the validity of *put*.

Theorem 1. *A put function is valid if and only if it satisfies* ViewDetermination *and* SourceStability.

Practically, there are few languages supporting putback-based bidirectional programming. This is not without reason: as argued by Foster [6], it is more difficult to construct a framework that can directly support putback-based bidirectional programming.

In the rest of this tutorial, we will introduce BiGUL [10] (which we pronounce as "beagle"), a simple yet powerful putback-based bidirectional language, which grew out of some prior putback-based languages [12,13]. BiGUL is implemented as an embedded language in Haskell, and we will assume that the reader is reasonably familiar with Haskell. After briefly explaining how to install BiGUL in Section 2, we will introduce basic BiGUL programming in Section 3, and see a few more examples about lists in Section 4. We will then move on to the underlying principles in Section 5, explaining the design and implementation of BiGUL in detail. The last three sections will show how various bidirectional applications can be developed, including list alignment in Section 6, relational database updating in Section 7, and parsing and "reflective" printing in Section 8.

2 Preparation: installing BiGUL

BiGUL has been released to Hackage. To install the latest version of BiGUL (1.0.1 at the time of writing), just run the following two commands:

```
> cabal update
> cabal install bigul
```

Now you can easily check whether BiGUL is correctly installed. First, create a simple file called "test.hs" with the following content for importing BiGUL modules.

```
{-# LANGUAGE FlexibleContexts, TemplateHaskell, TypeFamilies #-}
import Generics.BiGUL
import Generics.BiGUL.Interpreter
import Generics.BiGUL.TH
import Generics.BiGUL.Lib
```

Then load it using the GHC interactive interpreter.

```
> ghci test.hs
GHCi, version 7.10.3: http://www.haskell.org/ghc/  :? for help
[1 of 1] Compiling Main             ( test.hs, interpreted )
Ok, modules loaded: Main.
```

If you see the above message, congratulations on your successful installation.

3 A quick tour of BiGUL

Intuitively, we can think of a bidirectional BiGUL program

$$bx :: BiGUL\ s\ v$$

as describing how to manipulate a state consisting of a source component of type s and a view component of type v; the goal is to embed all information in the view to proper places in the source. For each $bx :: BiGUL\ s\ v$, we can run it forwards by calling *get* and backwards by calling *put*:

$$\begin{aligned} &get\ \ bx :: s \quad\ \ \rightarrow Maybe\ v \\ &put\ bx :: s \rightarrow v \rightarrow Maybe\ s \end{aligned}$$

Here, *get bx* is a function mapping a source to a view, but can possibly fail: it either returns a successfully computed view wrapped in the *Just* constructor of *Maybe*, or signifies failure by producing the *Nothing* constructor. On the other hand, *put bx* accepts an original source and uses a view to update it to get an updated source (and might fail as well).

In BiGUL, it suffices for the programmer to write the *put* behavior (i.e., how to use a view to update the original source to a new source), and the (unique) *get* behavior is obtained for free. The core of BiGUL consists of a small number of primitives and combinators for constructing well-behaved bidirectional transformations, which we introduce below.

3.1 Skip

The first primitive for writing *put* is

$$Skip :: (s \rightarrow v) \rightarrow BiGUL\ s\ v$$

The put behavior of *Skip f* keeps the source unchanged, provided that the view is computable from the source by f (while in the get direction, the view is fully computed by applying function f to the source). Consider a simple *put* defined by *Skip square* where

$$square\ x = x * x$$

We can test its put behavior as follows:

```
*PBasic> put (Skip square) 10 100
Just 10
```

It first checks if the view 100 is the square of the source 10. If that is the case, the original source is returned. But if the view is changed, say to 250, it should produce *Nothing*:

```
*PBasic> put (Skip square) 10 250
Nothing
```

To see why *put* produces *Nothing*, we may use *putTrace* instead of *put* to get more information:

```
*PBasic> putTrace (Skip square) 10 250
view not determined by the source
```

Each putback transformation in BiGUL is equipped with a unique *get* for doing forward transformation. We can test the *get* behavior as follows:

```
*PBasic> get (Skip square) 5
Just 25
```

In prose: doing the forward transformation of *Skip square* on the source 5 gives the view 25. If *get* fails, we can also use *getTrace* to see more information about the failure, analogous to *putTrace*.

As a simple exercise, can you see what the following *skip1* does?

$$skip1 :: BiGUL\ s\ ()$$
$$skip1 = Skip\ (const\ ())$$

3.2 Replace

The second primitive is

$$Replace :: BiGUL\ s\ s$$

which completely replaces the source with the view. For instance,

```
*PBasic> put Replace 1 100
Just 100
```

uses the view 100 to replace the source 1 and gets a new source 100.

3.3 Product

To use a view pair (v_1, v_2) to update a source pair (s_1, s_2), we can write *Prod bx1 bx2* or *bx1* `` `Prod` `` *bx2*, a product of two putback transformations *bx1* and *bx2*, to use v_1 to update s_1 with *bx1* and v_2 to s_2 with *bx2*.

$$Prod :: BiGUL\ s_1\ v_1 \to BiGUL\ s_2\ v_2 \to BiGUL\ (s_1, s_2)\ (v_1, v_2)$$

For instance, we can use *Prod* to combine *Skip* and *Replace* to put a view pair into a source pair.

```
*PBasic> put (skip1 `Prod` Replace) (5,1) ((),100)
Just (5,100)
```

Generally, we can use nested *Prod*s to describe a complicated structural mapping:

```
*PBasic> put ((skip1 `Prod` Replace) `Prod` Replace)
             ((5,1),2) (((),100),200)
Just ((5,100),200)
```

3.4 Source/view rearrangement

So far, the source and view are of the same structure. What if we wish to put a view (v_1, v_2) into a source of a different structure, say $((s_0, s_1), s_2)$, to replace s_1 by v_1 and s_2 by v_2? To do that, we need to rearrange the source and view into the same structure, and BiGUL provides a way of rearranging either the source or view through a "simple" λ-expression e:

$$
\begin{aligned}
&{}^{\$}(\mathit{rearrS}\ [\![\, e :: s_1 \to s_2 \,]\!]) :: \mathit{BiGUL}\ s_2\ v\ \to \mathit{BiGUL}\ s_1\ v \\
&{}^{\$}(\mathit{rearrV}\ [\![\, e :: v_1 \to v_2 \,]\!]) :: \mathit{BiGUL}\ s\ \ v_2 \to \mathit{BiGUL}\ s\ \ v_1
\end{aligned}
$$

The "simple" λ-expression e should be wrapped inside Template Haskell quasi-quotes $[\![\ldots]\!]$ (written as `[| ... |]` in plain text Haskell). By "simple" we mean that there should be no wildcards '_' in the argument pattern, and that the body can only contain the argument variables and constructors, and must mention all the argument variables. Returning to the problem of putting a pair into a triple, we may define the following putback transformation by rearranging the view:

$$
\begin{aligned}
&\mathit{putPairOverNPair} :: (\mathit{Show}\ s_0, \mathit{Show}\ s_1, \mathit{Show}\ s_2) \\
&\qquad\qquad \Rightarrow \mathit{BiGUL}\ ((s_0, s_1), s_2)\ (s_1, s_2) \\
&\mathit{putPairOverNPair} = {}^{\$}(\mathit{rearrV}\ [\![\, \lambda(v_1, v_2) \to (((), v_1), v_2) \,]\!])^{\$} \\
&\qquad\qquad (\mathit{skip1}\ `\mathit{Prod}`\ \mathit{Replace})\ `\mathit{Prod}`\ \mathit{Replace}
\end{aligned}
$$

The mechanism of source/view rearrangement enables us to process algebraic data structures such as lists and trees. The following example uses the view to replace the first element of a nonempty source list:

$$
\begin{aligned}
&\mathit{pHead} :: \mathit{Show}\ s \Rightarrow \mathit{BiGUL}\ [s]\ s \\
&\mathit{pHead} = {}^{\$}(\mathit{rearrS}\ [\![\, \lambda(s : ss) \to (s, ss) \,]\!])^{\$} \\
&\qquad\qquad {}^{\$}(\mathit{rearrV}\ [\![\, \lambda v \to (v, ()) \,]\!])^{\$} \\
&\qquad\qquad\quad \mathit{skip1}\ `\mathit{Prod}`\ \mathit{Replace}
\end{aligned}
$$

```
*PBasic> put pHead [1,2,3,4] 100
Just [100,2,3,4]
```

What if we wish to define a general putback transformation that uses the view to replace the ith element of the source list? We can define it recursively as follows:

$$
\begin{aligned}
&\mathit{pNth} :: \mathit{Show}\ s \Rightarrow \mathit{Int} \to \mathit{BiGUL}\ [s]\ s \\
&\mathit{pNth}\ i = \mathbf{if}\ i \equiv 0\ \mathbf{then}\ \mathit{pHead} \\
&\qquad\qquad\qquad \mathbf{else}\ \ {}^{\$}(\mathit{rearrS}\ [\![\, \lambda(x : xs) \to (x, xs) \,]\!])^{\$} \\
&\qquad\qquad\qquad\qquad {}^{\$}(\mathit{rearrV}\ [\![\, \lambda v \to ((), v) \,]\!])^{\$} \\
&\qquad\qquad\qquad\qquad\quad \mathit{skip1}\ `\mathit{Prod}`\ \mathit{pNth}\ (i - 1)
\end{aligned}
$$

```
*PBasic> put (pNth 3) [1..10] 100
Just [1,2,3,100,5,6,7,8,9,10]
```

As we know, any putback function in BiGUL is equipped with a *get* function. For *pNth*, we can test its *get* behavior as follows; its corresponding *get* function is actually the familiar *take* function.

```
*PBasic> get (pNth 3) [1..10]
Just 4
```

Both *pHead* and *pNth* contain the programming pattern in which both the source and view are rearranged into a product and then further updates are performed on corresponding components. This is a ubiquitous pattern in BiGUL, for which we provide a more compact syntax:

$$^{\$}(update\ ^{\mathbb{P}}[\![\, sourcePattern \,]\!]\ ^{\mathbb{D}}[\![\, viewPattern \,]\!]\ ^{\mathbb{D}}[\![\, updates \,]\!])$$

The source and view are respectively decomposed using *sourcePattern* and *viewPattern* inside the pattern quasi-quotes $^{\mathbb{P}}[\![\ldots]\!]$ (written as `[p| ... |]` in plain text Haskell), and corresponding elements are updated using the programs provided in the declaration quasi-quote $^{\mathbb{D}}[\![\ldots]\!]$ (`[d| ... |]` in plain text Haskell). For example, we may describe (*skip1* ‘*Prod*‘ *Replace*) ‘*Prod*‘ *Replace* by

$$\begin{aligned}
&testUpdate :: (Show\ a, Show\ b, Show\ c) \Rightarrow BiGUL\ ((a,b),c)\ (((),b),c)\\
&testUpdate = {}^{\$}(update\ {}^{\mathbb{P}}[\![\, ((x,y),z) \,]\!]\\
&\qquad\qquad\qquad\qquad {}^{\mathbb{P}}[\![\, ((x,y),z) \,]\!]\\
&\qquad\qquad\qquad\qquad {}^{\mathbb{D}}[\![\, x = skip1; y = Replace; z = Replace \,]\!])
\end{aligned}$$

In this concrete example, the three elements of the tuple (in both the source and view) are bound to the variables x, y, and z, and they are sent to the three combinators as arguments in the $^{\mathbb{D}}[\![\ldots]\!]$ part. Note that since *skip1* does nothing on its source but checks if its view is (), we can just match that source element with a wildcard ‘_’ in the source pattern and avoid writing *skip1* in $^{\mathbb{D}}[\![\ldots]\!]$.

$$\begin{aligned}
&testUpdate' :: (Show\ a, Show\ b, Show\ c) \Rightarrow BiGUL\ ((a,b),c)\ (((),b),c)\\
&testUpdate' = {}^{\$}(update\ {}^{\mathbb{P}}[\![\, ((_,y),z) \,]\!]\\
&\qquad\qquad\qquad\qquad {}^{\mathbb{P}}[\![\, (((),y),z) \,]\!]\\
&\qquad\qquad\qquad\qquad {}^{\mathbb{D}}[\![\, y = Replace; z = Replace \,]\!])
\end{aligned}$$

3.5 Case

The *Case* combinator is for case analysis, and the general structure is as follows:

$$\begin{aligned}
&Case\ [{}^{\$}(normal\ [\![\, mainCond1 :: s \to v \to Bool \,]\!]\ [\![\, exitCond1 :: s \to Bool \,]\!])\\
&\qquad \Longrightarrow (bx1 :: BiGUL\ s\ v)\\
&\quad , {}^{\$}(adaptive\ [\![\, mainCond1' :: s \to v \to Bool \,]\!])\\
&\qquad \Longrightarrow (f1 :: s \to v \to s)\\
&\quad , \ldots\\
&\quad , {}^{\$}(normal\ [\![\, mainCondn :: s \to v \to Bool \,]\!]\ [\![\, exitCondn :: s \to Bool \,]\!])
\end{aligned}$$

```
    ⟹ (bxn :: BiGUL s v)
  , ...
  , $(adaptive ⟦ mainCondn' :: s → v → Bool ⟧)
    ⟹ (fn :: s → v → s)
  ]
:: BiGUL s v
```

It contains a sequence of cases, each of which is either *normal* or *adaptive*. For a normal case, if the main condition is satisfied, a corresponding putback transformation is applied. For an adaptive case, if the main condition is satisfied, a function is used to produce an adapted source from the current source and view before the whole *Case* is rerun, with the expectation that one of the normal cases will be applicable this time. Note that if adaptation does not direct execution to a normal case, an error will be reported at runtime.

Note that $(*normal* ...) takes two predicates, which we call the *main condition* and the *exit condition*. The predicate for the main condition is very general, and we can use any function f of type $(s \to v \to Bool)$ to examine the source and view. If the condition is matched, then the BiGUL program after the predicate is executed. If the condition is not satisfied, the next branch is tried. The predicate for the exit condition checks the source only. The exit conditions in different branches should be disjoint (in principle).

As a simple example, consider using the view to update all the elements in the source list. To do so, we use *Case* to describe a case analysis.

```
replaceAll :: (Eq s, Show s) ⇒ BiGUL [s] s
replaceAll =
  Case [$(normal ⟦ λs v → length s ≡ 1 ⟧ ⟦ λs → length s ≡ 1 ⟧)
          ⟹ $(rearrS ⟦ λ[x] → x ⟧) Replace
       , $(normal ⟦ λs v → length s > 1 ⟧ ⟦ λs → length s > 1 ⟧)
          ⟹ $(rearrS ⟦ λ(x : xs) → (x, xs) ⟧)$
                $(rearrV ⟦ λv → (v, v) ⟧)$
                  Replace `Prod` replaceAll,
       , $(adaptive ⟦ λs v → length s ≡ 0 ⟧)
          ⟹ λs v → [⊥]
       ]
```

```
*PBasic> put replaceAll [] 100
Right [100]
*PBasic> put replaceAll [1..10] 100
Right [100,100,100,100,100,100,100,100,100,100]
```

Note that instead of using a general function to describe the main condition or the exit condition, we allow to use patterns. The syntax for the normal and the adaptive cases are:

```
$(normalSV ℙ⟦ sourcePattern ⟧ ℙ⟦ viewPattern ⟧ ⟦ exitCond ⟧)
  ⟹ (bx :: BiGUL s v)
```

$$^{\$}(adaptiveSV\ ^{\mathbb{P}}[\![\ sourcePattern\ ]\!]\ ^{\mathbb{P}}[\![\ viewPattern\ ]\!]) \\ \Longrightarrow (f :: s \to v \to s)$$

3.6 View dependency

Sometimes, a view may contain derived values that are computed from other part of the view, and are not allowed to be changed. For instance, for the view $(x, x+1)$, the second component is computed from the first by increasing it by 1. To capture this, BiGUL provides

$$Dep :: Eq\ v' \Rightarrow (v \to v') \to BiGUL\ a\ v \to BiGUL\ a\ (v, v')$$

to describe this intention. We may, for example, define

$$replaceAll2 :: BiGUL\ [Int]\ (Int, Int) \\ replaceAll2 = Dep\ (+1)\ replaceAll$$

to replace all elements of the source by the first component of the view, while the second component of the view is a derived one that should be one larger than the first one.

```
*PBasic> put replaceAll2 [1..10] (100,101)
Just [100,100,100,100,100,100,100,100,100,100]
*PBasic> put replaceAll2 [1..10] (100,200)
Nothing
*PBasic> putTrace replaceAll2 [1..10] (100,200)
second view component not determined by the first
```

As seen in the last running of *put*, it reports an error because in the view $(100, 200)$, 200 is not one larger than 100.

3.7 Composition

Bidirectional transformations can be composed.

$$Compose :: BiGUL\ a\ u \to BiGUL\ u\ b \to BiGUL\ a\ b$$

As a simple example, we define the following *pHead2* to use the view to update the first element of the first element of the source.

$$pHead2 :: Show\ a \Rightarrow BiGUL\ [[a]]\ a \\ pHead2 = pHead\ `Compose`\ pHead$$

```
*PBasic> put pHead2 [[1,2],[3,4,5],[]] 100
Just [[100,2],[3,4,5],[]]
```

3.8 Utilities

`Generics.BiGUL.Lib` has some useful predefined functions for building putback transformations. An interesting one is *emb*, which can safely embed a pair of well-behaved *get* and *put* into BiGUL. *emb* itself is defined as follows:

$$
\begin{aligned}
&emb :: Eq\ v \Rightarrow (s \to v) \to (s \to v \to s) \to BiGUL\ s\ v \\
&emb\ g\ p = Case \\
&\quad [\,^{\$}(normal\ [\![\, \lambda s\ v \to g\ s \equiv v \,]\!]^{\mathbb{P}}\ [\![\, \_ \,]\!]) \\
&\qquad \Longrightarrow Skip\ g \\
&\quad ,\,^{\$}(adaptive\ [\![\, \lambda \_\ \_ \to otherwise \,]\!]) \\
&\qquad \Longrightarrow p \\
&\quad]
\end{aligned}
$$

As an application of *emb*, we may define the following putback function to use a sum to update a pair.

$$
\begin{aligned}
&distSum :: BiGUL\ (Int, Int)\ Int \\
&distSum = emb\ g\ p \\
&\quad \textbf{where}\ g\ (x, y) = x + y \\
&\qquad\quad\ \ p\ (x, y)\ v = (v - y, y)
\end{aligned}
$$

```
*PBasic> put distSum (1,2) 100
Right (98,2)
*PBasic> get distSum (1,2)
Right 3
```

4 Bidirectional programming on lists

In this section, we demonstrate that many list functions can be bidirectionalized using BiGUL. To show the correspondence with the original list functions, we prefix the original forward function names with "lens". Note that in our context, the original forward functions can be automatically derived from the new putback transformations by calling *get*.

We shall focus on bidirectionalizing *foldr*, a simple but useful higher-order function on lists:

$$
\begin{aligned}
&foldr\ f\ e\ [\,] \quad\ \ = e \\
&foldr\ f\ e\ (x : xs) = f\ x\ (foldr\ f\ e\ xs)
\end{aligned}
$$

where many interesting functions can be defined in terms of *foldr*:

$$
\begin{aligned}
&sum \quad\ \ = foldr\ (+)\ 0 \\
&map\ f \quad = foldr\ (\lambda a\ r \to f\ a : r)\ [\,] \\
&filter\ p \quad = foldr\ (\lambda a\ r \to \textbf{if}\ p\ a\ \textbf{then}\ a : r\ \textbf{else}\ r)\ [\,] \\
&reverse\ p = foldr\ (\lambda a\ r \to r +\!\!+ [a])\ [\,] \\
&sort \quad\ \ = foldr\ insert\ [\,]
\end{aligned}
$$

We start by developing a putback version for *foldr*.

```
lensFoldr :: (Show a, Show b)
          ⇒ BiGUL (a, b) b → (b → Bool) → BiGUL ([a], b) b
lensFoldr bx pv =
  Case [$(adaptive [[ λ(x, y) v → pv v ∧ length x ≢ 0 ]])
          ⟹ λ(x, y) v → ([], y)
       ,$(normal [[ λ(xs, _) v → null xs ]] [[ λ(xs, _) → null xs ]])
          ⟹ $(rearrV [[ λv → ((), v) ]])$
               $(update ℙ[[ (_, v) ]] ℙ[[ ((), v) ]] 𝔻[[ v = Replace ]])
       ,$(normalSV ℙ[[ _ ]] ℙ[[ _ ]] [[ λ(xs, _) → not (null xs) ]])
          ⟹ $(rearrS [[ λ((x : xs), e) → (x, (xs, e)) ]])$
               (Replace `Prod` lensFoldr bx pv) `Compose` bx
       ]
```

The above *lensFoldr bx pv* updates source (xs, y) with view v. If v satisfies pv, it will try to stop by adapting xs to $[\,]$. Or it will recursively apply bx rightwards over the elements of xs: if xs is empty, it will just embed v to y; otherwise it will rearrange the source for the recursive call.

With *lensFoldr*, we can redefine many list functions from the putback point of view. As the first example, *map* can be defined in terms of *lensFoldr*.

```
lensMap :: (Show a, Show b) ⇒ BiGUL a b → BiGUL ([a], [b]) [b]
lensMap bx = lensFoldr bx' null
  where bx' = $(rearrV [[ λ(v : vs) → (v, vs) ]])$
                bx `Prod` Replace
```

```
*PList> put (lensMap dec1) ([0..10],[]) [100..110]
Just ([99,100,101,102,103,104,105,106,107,108,109],[])
*PList> get (lensMap dec1) ([1..10],[])
Just [2,3,4,5,6,7,8,9,10,11]
```

Here, for testing, we embed into our framework the bijective functions for increasing and decreasing an integer by 1.

```
dec1 :: BiGUL Int Int
dec1 = emb g p
  where g s = s + 1
        p s v = v − 1
```

The second example is to bidirectionalize *reverse*, which is to reverse the elements of a list.

```
lensReverse :: Show a ⇒ BiGUL [a] [a]
lensReverse =
  Case [$(adaptive [[ λs v → length s < length v ]])
```

$$
\begin{array}{l}
\quad \Longrightarrow \lambda s\ v \to v \\
, {}^{\$}(normalSV\ {}^{\mathbb{P}}[\![\_]\!]\ {}^{\mathbb{P}}[\![\_]\!]\ [\![\lambda s \to True]\!]) \\
\quad \Longrightarrow {}^{\$}(rearrS\ [\![\lambda s \to (s, [\,])]\!])^{\$} \\
\qquad\qquad lensFoldr\ (lensSwap\ `Compose`\ lensSnoc)\ null \\
]
\end{array}
$$

When the source is shorter than the view, we add more elements to the source by filling in it with the elements of the view (Note that this is one option; we could do other ways by, say, duplicating elements in the source). When the source is long enough, we decompose the view into a snoc-list using *lensSnoc*, and use it to update the cons-list of the source.

$$
\begin{array}{l}
lensSnoc :: Show\ a \Rightarrow BiGUL\ ([a], a)\ [a] \\
lensSnoc = \\
\quad Case\ [{}^{\$}(normal\ [\![\lambda s\ v \to length\ v \equiv 1]\!]\ [\![\lambda(s, \_) \to null\ s]\!]) \\
\qquad \Longrightarrow {}^{\$}(rearrV\ [\![\lambda[v] \to ([\,], v)]\!])\ Replace \\
\qquad , {}^{\$}(normal\ [\![\lambda(s, \_)\ v \to length\ s > 0]\!]\ [\![\lambda(s, \_) \to length\ s > 0]\!]) \\
\qquad \Longrightarrow {}^{\$}(rearrS\ [\![\lambda(y : ys, x) \to (y, (ys, x))]\!])^{\$} \\
\qquad\quad {}^{\$}(rearrV\ [\![\lambda(v : vs) \to (v, vs)]\!])^{\$} \\
\qquad\qquad Replace\ `Prod`\ lensSnoc \\
\qquad , {}^{\$}(adaptive\ [\![\lambda(s, \_)\ v \to null\ s]\!]) \\
\qquad \Longrightarrow \lambda(s, x)\ \_ \to ([\bot], x) \\
\qquad] \\
lensSwap :: (Show\ a, Show\ b) \Rightarrow BiGUL\ (a, b)\ (b, a) \\
lensSwap = {}^{\$}(rearrS\ [\![\lambda(x, y) \to (y, x)]\!])\ Replace
\end{array}
$$

Below are some testing examples.

```
*PList> put lensSnoc ([2,3,4],1) [10,11,12,13]
Just ([10,11,12],13)
*PList> put lensSnoc ([2,3,4],1) [10,11,12,13,14]
Just ([10,11,12,13],14)
*PList> put lensSnoc ([2,3,4],1) [10,11]
Just ([10],11)
*PList> get lensSnoc ([1..10], 100)
Just [1,2,3,4,5,6,7,8,9,10,100]

*PList> put lensReverse [1..10] [100..105]
Just [105,104,103,102,101,100]
*PList> put lensReverse [1..10] [100..115]
Just [115,114,113,112,111,110,109,108,107,106,105,104,103,102,101,100]
*PList> get lensReverse [1..10]
Just [10,9,8,7,6,5,4,3,2,1]
```

It is worth noting that our definition of *lensFoldr* is just one candidate put-back function for *foldr*, and there are many others. This reflects the fact that one *foldr* can have many *put*s, each describing different updating strategy.

5 BiGUL's Bidirectionality

We have been writing *put* programs, usually having a corresponding *get* in mind but not explicitly describing it, and yet BiGUL is capable of finding the right *get* behaviour as if it could read our mind. How? We will see that, when writing a BiGUL program, we are always simultaneously describing both a *put* function and a *get* function, which are guaranteed to be a well-behaved pair. And the "mind-reading" ability is far from magic: It is the consequence of the fact that well-behavedness directly implies that *get* is uniquely determined by *put*, which is the main motivation for designing a putback-based language. In this section, we will first review the theory, this time explicitly taking *partiality* into account, and then we will dive into BiGUL's internals to get a taste of putback-based design.

5.1 Lenses, well-behavedness, and the fundamental theorem

Formally, we call a well-behaved pair of *put* and *get* a *lens*:

Definition 2 (lens). *A* lens *between a source type s and a view type v consists of two functions:*

$$\begin{aligned} &put :: s \to v \to Maybe\ s \\ &get :: s \quad\ \ \to Maybe\ v \end{aligned}$$

satisfying two well-behavedness laws:

$$put\ s\ v = Just\ s' \quad \Rightarrow \quad get\ s' = Just\ v \qquad \text{(PutGet)}$$

$$get\ s = Just\ v \quad \Rightarrow \quad put\ s\ v = Just\ s \qquad \text{(GetPut)}$$

In the original formulation [7], a lens refers to just a pair of functions having the right types, and one needs to explicitly say "well-behaved lens" to mean a well-behaved pair; we will, however, discuss well-behaved lenses only, so we build well-behavedness into our definition of lenses by default. Also note that this definition models partial transformations explicitly as *Maybe*-valued functions: *put* and *get* are *total* functions that can nevertheless produce *Nothing* to indicate failure.

From this definition of well-behavedness, we can immediately prove what might be called the "fundamental theorem" of putback-based bidirectional programming:

Theorem 2 (uniqueness of *get*). *Given two lenses whose put components are equal, their get components are also equal.*

Proof. Let l and r be two lenses; denote their put/get components as $put\ l/get\ l$ and $put\ r/get\ r$ respectively, and assume that $put\ l = put\ r$. Then for any

s and v,

$$
\begin{array}{ll}
 & \mathit{get}\ l\ s = \mathit{Just}\ v \\
\Leftrightarrow & \{\ \text{well-behavedness of } l\ \} \\
 & \mathit{put}\ l\ s\ v = \mathit{Just}\ s \\
\Leftrightarrow & \{\ \mathit{put}\ l = \mathit{put}\ r\ \} \\
 & \mathit{put}\ r\ s\ v = \mathit{Just}\ s \\
\Leftrightarrow & \{\ \text{well-behavedness of } r\ \} \\
 & \mathit{get}\ r\ s = \mathit{Just}\ v
\end{array}
$$

(This also entails that $\mathit{get}\ l\ s = \mathit{Nothing}$ if and only if $\mathit{get}\ r\ s = \mathit{Nothing}$.) □

The theorem guarantees that the BiGUL programmer is in full control of the bidirectional behavior — programming the *put* behavior is sufficient to determine the *get* behavior. Also, to the language designer, the theorem gives a kind of reassurance that, once the *put* behavior of a construct is determined, there is no need to worry about which *get* behavior should be adopted — there is at most one possibility. This is in contrast to *get*-based design, in which there are usually more than one viable *put* semantics that can be assigned to a *get*-based construct, and the designer needs to justify the choice or provide several versions.

For the rest of this section, we will look at several constructs of BiGUL in detail to get a taste of putback-based design. Each BiGUL construct is conceived, at the design stage, as a lens (like *Skip* and *Replace*) or a lens *combinator* (like *Case*), which constructs a more complex lens from simpler ones. The *put* and *get* components of these lenses usually have to be developed together, but for each lens we will employ a more "*put*-oriented" design process: We start from an intended *put* behavior, and then add restrictions so that we can find a corresponding *get*. This does not guarantee that the lenses we arrive at will have a "strong *put* flavor" — that is, some of the lenses will be as (or even more) suitable for *get*-based programming as for putback-based programming. But we will also see that some other lenses are more naturally understood in terms of their *put* behavior.

5.2 Replacement

The simplest lens is probably *Replace*, which replaces the entire source with the view:

$$\mathit{put}\ \mathit{Replace}\ s\ v = \mathit{Just}\ v$$

Is there a *get* semantics that can be paired with this *put*? Yes, quite obviously — in fact, PutGet directly gives us the definition of *get Replace*:

$$\mathit{get}\ \mathit{Replace}\ v = \mathit{Just}\ v$$

We still need to verify GetPut, which can be easily checked to be true.

5.3 Skipping

Coming up next is *Skip*, whose natural behavior is

$$put\ Skip\ s\ v = Just\ s$$

Considering PutGet, though, we immediately see that this behavior is too liberal: If the view is simply thrown away, how can *get Skip* possibly recover it? One way out is to require that the view is trivial enough such that it can be thrown away and still be recovered, by setting the view type of *Skip* to the unit type $()$. Then it is easy for *get Skip* to recover the view, for which there is only one choice:

$$get\ Skip\ s = Just\ ()$$

This is the approach adopted prior to BiGUL 1.0.

More generally, we can establish well-behavedness as long as *get Skip* has only one view choice for each source, regardless of what the view type is. The existence of this "unique choice" is witnessed by a function $f :: s \to v$, which we add as an additional argument to *Skip*. The *get* direction is then

$$get\ (Skip\ f)\ s = Just\ (f\ s)$$

From the *put* direction, we may think of this function f as specifying a consistency relation, saying that the view information is completely included in the source (since you can compute the view from the source) and can be safely discarded. *Skip* f can be used if and only if the source and view are consistent in that sense. We thus arrive at:

$$put\ (Skip\ f)\ s\ v = \textbf{if}\ v \equiv f\ s\ \textbf{then}\ return\ s\ \textbf{else}\ Nothing$$

This pair of *put* and *get* can be verified to be well-behaved. *Skip* f, which features in BiGUL 1.0, is one lens which turns out to be more easily understood from the *get* direction — it bidirectionalizes any *get* function whose codomain has decidable equality, albeit trivially. We recover the first version of *Skip* as a special case by setting f to $const\ ()$.

5.4 Product

For a simplest example of a lens combinator, we look at *Prod*. Both the source and view types should be pairs; *Prod* accepts two lenses, say l and r, and applies them respectively to the left and right components:

$$\begin{aligned} put\ (l\ `Prod`\ r)\ (sl, sr)\ (vl, vr) = \textbf{do}\ & sl' \leftarrow put\ l\ \ sl\ \ vl \\ & sr' \leftarrow put\ r\ sr\ vr \\ & return\ (sl', sr') \end{aligned}$$

The *get* direction is unsurprising:

$$\begin{aligned} get\ (l\ `Prod`\ r)\ (sl, sr) = \mathbf{do}\ & vl \leftarrow get\ l\ \ sl \\ & vr \leftarrow get\ r\ sr \\ & return\ (vl, vr) \end{aligned}$$

Having constructed *put* and *get* from l and r, we also expect that their well-behavedness is a consequence of the well-behavedness of l and r. Intuitively, this pair of *put* and *get* does look well-behaved if l and r are. But how do we establish this formally? To prove PutGet, for example, we should prove that the assumption

$$put\ (l\ `Prod`\ r)\ (sl, sr)\ (vl, vr) = Just\ (sl', sr') \tag{1}$$

implies the conclusion

$$get\ (l\ `Prod`\ r)\ (sl', sr') = Just\ (vl, vr) \tag{2}$$

Both equations say that a somewhat complicated monadic *Maybe*-program computes successfully to some value. It may seem that we need some messy case analysis, but what we know about *Maybe*-programs tells us that such a program computes successfully if and only if every step of the program does, and this helps us to split both (1) and (2) into simpler equations. Formally, we have this lemma:

Lemma 2. *Let* $mx :: Maybe\ a$ *and* $f :: a \to Maybe\ b$*. Then, for all* $y :: b$*,*

$$mx \gg\!= f = Just\ y$$

if and only if

$$mx = Just\ x \quad and \quad f\ x = Just\ y \qquad for\ some\ x :: a$$

Proof. Case analysis on mx. □

This lemma can be nicely applied to *Maybe*-programs written in the **do**-notation, transforming such programs into *predicates* saying that a program computes to some given value. To do it more formally: Define a translation $\mathcal{S}$ from **do**-blocks of type $Maybe\ a$ to predicates on a by

$$\begin{aligned} \mathcal{S}\ (\mathbf{do}\ \{x \leftarrow mx; B\})\ y &= \exists x.\ mx = Just\ x \wedge \mathcal{S}\ (\mathbf{do}\ B)\ y \\ \mathcal{S}\ (\mathbf{do}\ \{my\})\ y &= my = Just\ y \end{aligned}$$

Then we can extend Lemma 2 to the following:

Lemma 3. *The proposition*

$$\mathcal{S}\ (\mathbf{do}\ B)\ y$$

is true if and only if

$$\mathbf{do}\ B = Just\ y$$

Proof. By induction on the list structure of B, using Lemma 2 repeatedly. □

For example, applying $\mathcal{S}$ to $put\ (l\ `Prod`\ r)\ (sl, sr)\ (vl, vr)$ yields

$$\begin{aligned}\lambda(sl'', sr''). \exists sl'.\ & put\ l\ sl\ vl = Just\ sl' \wedge \\ \exists sr'.\ & put\ r\ sr\ vr = Just\ sr' \wedge \\ & return\ (sl', sr') = Just\ (sl'', sr'')\end{aligned}$$

where the last equation is equivalent to $sl' = sl'' \wedge sr' = sr''$ (since $return = Just$ for the *Maybe* monad). Applying Lemma 3 and doing some simplification, (1) is equivalent to

$$put\ l\ sl\ vl = Just\ sl' \quad \wedge \quad put\ r\ sr\ vr = Just\ sr'$$

Similarly, (2) can be shown to be equivalent to

$$get\ l\ sl' = Just\ vl \quad \wedge \quad get\ r\ sr' = Just\ vr$$

The entailment is then just PutGet for l and r.

5.5 Case analysis

This is a representative combinator in BiGUL, and arguably the most complex one. For simplicity, let us consider a two-branch variant of *Case*. A branch is a condition and a body; since in *put* we manipulate both a source and a view, the conditions in general can be binary predicates on both the source and view. We thus define the type of branches as:

$$\textbf{type}\ CaseBranch\ s\ v = (s \to v \to Bool, BiGUL\ s\ v)$$

and consider the following variant of *Case*:

$$Case :: CaseBranch\ s\ v \to CaseBranch\ s\ v \to BiGUL\ s\ v$$

The straightforward behavior is

$$\begin{aligned}put\ (Case\ (pl, l)\ (pr, r))\ s\ v = \ & \textbf{if}\ \ pl\ \ s\ v\ \textbf{then}\ put\ l\ \ s\ v \\ & \textbf{else if}\ pr\ s\ v\ \textbf{then}\ put\ r\ s\ v \\ & \textbf{else}\ Nothing\end{aligned}$$

That is, depending on which condition is satisfied (with pl having higher priority), we execute either $put\ \ l$ or $put\ \ r$, or fail the computation if neither of the conditions is satisfied. Now, again, we ask the question: Can we find a *get* behavior to pair with this *put*?

Ruling out branch switching for PutGet. An important working assumption here is that we want lens combinators to be *compositional*: When we looked at *Prod*, for example, we defined its *put* and *get* in terms of those of the smaller lenses, and derived the overall well-behavedness from that of the smaller lenses.

For *Case*, this implies that, when establishing well-behavedness, we want a *get* following a *put* (or a *put* following a *get*) to use the same branch taken by the *put* (or the *get*), so we can make use of PUTGET (or GETPUT) of the branch. The current *put* behavior of *Case* does not leave any clue in the updated source about which branch is used to produce it, though, so it is impossible for *get* to always choose the right branch.

One solution, which does not require changing the syntax of *Case*, is to check that the ranges of the branches are *disjoint*. In general, for a lens, the range of a *put* can be shown to coincide with the domain of the corresponding *get*. So the *get* behavior of *Case* can simply try to execute both branches on the input source, and there will be at most one branch that computes successfully. We can put (expensive) disjointness checks into *put* such that if *put* succeeds, the subsequent *get* will have at most one branch to choose:

$$
\begin{array}{l}
put\ (Case\ (pl, l)\ (pr, r))\ s\ v = \\
\quad \textbf{if} \quad pl\ \ s\ v\ \textbf{then do}\ s' \leftarrow put\ l\ \ s\ v \\
\qquad\qquad\qquad\qquad\quad maybe\ (return\ s')\ (const\ Nothing)\ (get\ r\ s') \\
\quad \textbf{else if}\ pr\ s\ v\ \textbf{then do}\ s' \leftarrow put\ r\ s\ v \\
\qquad\qquad\qquad\qquad\quad maybe\ (return\ s')\ (const\ Nothing)\ (get\ l\ \ s') \\
\quad \textbf{else}\ Nothing
\end{array}
$$

If *get* favors the first branch, meaning that it declares success as soon as the first branch succeeds (without requiring that the second branch fails),

$$get\ (Case\ (pl, l)\ (pr, r))\ s = maybe\ (get\ r\ s)\ return\ (get\ l\ s)$$

then we can also omit the check in *put*'s first branch:

$$
\begin{array}{l}
put\ (Case\ (pl, l)\ (pr, r))\ s\ v = \\
\quad \textbf{if} \quad pl\ \ s\ v\ \textbf{then}\ put\ l\ s\ v \\
\quad \textbf{else if}\ pr\ s\ v\ \textbf{then do}\ s' \leftarrow put\ r\ s\ v \\
\qquad\qquad\qquad\qquad\quad maybe\ (return\ s')\ (const\ Nothing)\ (get\ l\ s') \\
\quad \textbf{else}\ Nothing
\end{array}
$$

Ruling out branch switching for GetPut. The GETPUT direction, on the other hand, still does not avoid branch switching — the outcome of *get* does not say anything about which of *pl* and *pr* will be satisfied in the subsequent *put*. So we add some checks to *get* such that *get*'s success will tell us which branch will be chosen by *put*:

$$
\begin{array}{l}
get\ (Case\ (pl, l)\ (pr, r))\ s = maybe\ (\textbf{do}\ v \leftarrow getBranch\ (pr, r)\ s \\
\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\quad \textbf{if}\ pl\ s\ v\ \textbf{then}\ Nothing \\
\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\quad \textbf{else}\ \ return\ v) \\
\qquad\qquad\qquad\qquad\qquad\quad return \\
\qquad\qquad\qquad\qquad\qquad\quad (getBranch\ (pl, l)\ s) \\
getBranch\ (p, b)\ s = \textbf{do}\ v \leftarrow get\ b\ s
\end{array}
$$

```
                    if p s v then return v
                             else  Nothing
```

The definition of *put* should also be revised to use *getBranch* for disjointness check. This fixes GetPut, but breaks PutGet! Since *put* does not guarantee that the *updated* (not the original) source and the view satisfy the condition of the branch executed, even though *get* will be able to choose the right branch, the subsequent, newly added check is not guaranteed to succeed. We thus also need to add similar checks to *put*:

```
put (Case (pl, l) (pr, r)) s v =
  if      pl s v then do s' ← put l s v
                         if pl s' v then return s'
                                    else  Nothing
  else if pr s v then do s' ← put r s v
                         if pr s' v then maybe (return s')
                                               (const Nothing)
                                               (getBranch (pl, l) s')
                                    else  Nothing
  else Nothing
```

Now this pair of *put* and *get* can be verified to be well-behaved.

Improving the efficiency of *get*. The efficiency of the current *get* does not look very good, especially when, in general, an arbitrary number of branches are allowed, and *get* has to try to execute each branch, possibly with a high cost, until it reaches a successful one; also, inefficient *get* affects the efficiency of *put*, which calls *get* to check range disjointness. An idea is to ask the programmer to make a rough "prediction" of the range of each branch: We enrich *CaseBranch* with a third component, which is a source predicate:

```
type CaseBranch s v = (s → v → Bool, BiGUL s v, s → Bool)
```

This new predicate is supposed to be satisfied by the updated source; we again add checks to *put* to ensure this:

```
put (Case (pl, l, ql) (pr, r, qr)) s v =
  if      pl s v then do s' ← put l s v
                         if pl s' v ∧ ql s' then return s'
                                            else  Nothing
  else if pr s v then do s' ← put r s v
                         if pr s' v ∧ qr s'
                           then maybe (return s')
                                      (const Nothing)
                                      (getBranch (pl, l, ql) s')
                           else  Nothing
  else Nothing
```

Let us call *pl* and *pr* the *main conditions*, and *ql* and *qr* the *exit conditions*. The exit condition, in general, over-approximates the range of a branch. Well-behavedness tells us that the range of *put* is exactly the domain of the corresponding *get*. Thus, in the *get* direction, every source in the domain of a branch satisfies the exit condition. Contrapositively, if a source does not satisfy the exit condition, then *get* for that branch will necessarily fail, and we do not need to try to execute the branch at all. This leads to the following revised definition of *getBranch*:

$$\begin{aligned} &\mathit{getBranch}\ (\mathit{pl}, l, \mathit{ql})\ s = \mathbf{if}\ \mathit{ql}\ s\ \mathbf{then\ do}\ v \leftarrow \mathit{get}\ l\ s \\ &\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad \mathbf{if}\ \mathit{pl}\ s\ v\ \mathbf{then}\ \mathit{return}\ v \\ &\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\quad \mathbf{else}\ \ \mathit{Nothing} \\ &\qquad\qquad\qquad\qquad\qquad\qquad\quad \mathbf{else}\ \ \mathit{Nothing} \end{aligned}$$

If we do not care about efficiency, we can simply use *const True* as exit conditions, and the behavior will be exactly the same as the previous version. But if we supply disjoint exit conditions, then *get* will try at most one branch. Incidentally (but actually no less importantly), making exit conditions explicit also encourages the programmer to think about range disjointness, which is essential to guaranteeing the totality of *Case*.

Adaptation. We have seen that, to make *Case* total, one thing we need to ensure is that the main condition of a branch should be satisfied again after the update. In practice, the main condition is usually closely related to the consistency relation, and we will only be able to deal with sources and views that are already more or less consistent; this is a rather severe restriction. As we have seen in Section 3.5, the solution is to introduce a different kind of branches called *adaptive branches*, which can deal with sources and views that are too inconsistent by adapting the source to establish enough consistency such that a normal branch becomes applicable. Again, for simplicity, we consider only a variant of *Case* which has just one adaptive branch at the end:

$$\begin{aligned} &\mathbf{type}\ \mathit{CaseAdaptiveBranch}\ s\ v = (s \to v \to \mathit{Bool}, s \to v \to s) \\ &\mathit{Case} :: \mathit{CaseBranch}\ s\ v \to \mathit{CaseBranch}\ s\ v \to \\ &\qquad \mathit{CaseAdaptiveBranch}\ s\ v \to \mathit{BiGUL}\ s\ v \end{aligned}$$

The execution structure of *put* becomes slightly more complicated, as the whole thing has to be run again after adaptation; to ensure termination, we require that the second run do not match an adaptive branch again. This is realized in BiGUL in continuation-passing style:

$$\begin{aligned} &\mathit{put}\ (\mathit{Case}\ \mathit{bl}\ \mathit{br}\ \mathit{ba})\ s\ v = \\ &\quad \mathit{putWithAdaptation}\ \mathit{bl}\ \mathit{br}\ \mathit{ba}\ s\ v\ (\lambda \mathit{sa} \to \\ &\qquad \mathit{putWithAdaptation}\ \mathit{bl}\ \mathit{br}\ \mathit{ba}\ \mathit{sa}\ v\ (\mathit{const}\ \mathit{Nothing})) \\ &\mathit{putWithAdaptation} :: \\ &\quad \mathit{CaseBranch}\ s\ v \to \mathit{CaseBranch}\ s\ v \to \mathit{CaseAdaptiveBranch}\ s\ v \to \end{aligned}$$

```
  s → v → (s → Maybe s) → Maybe s
putWithAdaptation (pl, l, ql) (pr, r, qr) (pa, f) s v cont =
  if       pl s v then do s' ← put l s v
                          if pl s' v ∧ ql s' then return s'
                                             else Nothing
  else if pr s v then do s' ← put r s v
                          if pr s' v ∧ qr s'
                            then maybe (return s')
                                       (const Nothing)
                                       (getBranch (pl, l, ql) s')
                            else Nothing
  else if pa s v then cont (f s v)
  else Nothing
```

What about *get*? It turns out that *get* can simply ignore the adaptive branch! If you have doubt about this "choice," just invoke the fundamental theorem (Theorem 2): The *put* behavior is exactly what we want, and we can verify that the pair of *put* and *get* is well-behaved, so we are reassured that our "choice" is "correct," simply because there is no other choice of *get*.

To sum up, we have arrived at a simpler variant of *Case* which nevertheless has all the features of the multi-branch *Case* in BiGUL. We have inserted various dynamic checks into the *put* semantics, and the BiGUL programmer needs to be aware of these constraints to make execution of *Case* succeed: For each normal branch, (i) the main condition should be satisfied after the update, (ii) the main conditions of the branches before this one should not be satisfied after the update, and (iii) the exit condition should be satisfied by the updated source. Also the ranges of all the normal branches should be disjoint; the programmer is encouraged to write disjoint exit conditions, which imply disjointness of the ranges, and improve the efficiency of *get*. Finally, for each adaptive branch, the adapted source and the view should match the main condition of a normal branch.

5.6 Rearrangement

Source and view rearrangements are also among the more complex constructs of BiGUL. Their complexity lies in the strongly and generically typed treatment of pattern matching, though, rather than their bidirectional behavior. The two kinds of rearrangement are similar, and we will discuss view rearrangement only.

Pattern matching is inherently a bidirectional operation: In one direction, we break something into a collection of its components at the variable positions of a pattern. This collection can be considered as indexed by the variable positions, and acting like an *environment* for expression evaluation. Indeed, conversely, if we have a pattern and a corresponding environment, we can treat the pattern as an expression and evaluate it in the environment. These two directions are inverse to each other, i.e., they form a (partial) isomorphism. For the language designer, it may be slightly tedious to establish such isomorphisms, but for the

programmer, pattern matching and evaluation are arguably the most natural way to decompose and rearrange things. Previous bidirectional languages usually provide theoretically simpler combinators for decomposition and rearrangement, but they are hard to use in practice. BiGUL's native support of pattern matching, on the other hand, turns out to be one important contributing factor in its usability.

BiGUL's patterns are strongly typed: The programmer has to declare a target type for a pattern, and the pattern is guaranteed, through typechecking, to make sense for that target type. This can be achieved by defining the datatype of patterns as a generalised algebraic datatype:

$$
\begin{aligned}
&\textbf{data}\ \mathit{Pat}\ a\ \textbf{where}\\
&\quad \mathit{PVar}\ \ :: \mathit{Eq}\ a \Rightarrow \mathit{Pat}\ a\\
&\quad \mathit{PConst} :: \mathit{Eq}\ a \Rightarrow a \to \mathit{Pat}\ a\\
&\quad \mathit{PProd}\ \ :: \mathit{Pat}\ a \to \mathit{Pat}\ b \to \mathit{Pat}\ (a, b)\\
&\quad \mathit{PLeft}\ \ :: \mathit{Pat}\ a \to \mathit{Pat}\ (\mathit{Either}\ a\ b)\\
&\quad \mathit{PRight}\ :: \mathit{Pat}\ b \to \mathit{Pat}\ (\mathit{Either}\ a\ b)\\
&\quad \mathit{PIn}\ \ \ :: \mathit{InOut}\ a \Rightarrow \mathit{Pat}\ (F\ a) \to \mathit{Pat}\ a
\end{aligned}
$$

A pattern can be a (nameless) variable, a constant, a product, a *Left* or *Right* injection (for the *Either* type), or a generic constructor, and its target type is given as the index in its type. For example, the pattern $\mathit{PLeft}\ (\mathit{PConst}\ ())$ has type $\mathit{Pat}\ (\mathit{Either}\ ()\ b)$, and can only be used to match those values of type $\mathit{Either}\ ()\ b$ (and matching succeeds only for the value $\mathit{Left}\ ()$). The *InOut* typeclass contains the types that are isomorphic to (and therefore interconvertible with) a sum-of-products representation. The isomorphism is witnessed by

$$\mathit{inn} :: \mathit{InOut}\ a \Rightarrow F\ a \to a \qquad \text{and} \qquad \mathit{out} :: \mathit{InOut}\ a \Rightarrow a \to F\ a$$

which will be used to define pattern matching and evaluation. For example, $[a]$ is an instance of *InOut*, and $F\ [a]$, an isomorphic sum-of-products representation of $[a]$, is $\mathit{Either}\ ()\ (a, [a])$. The two functions witnessing the isomorphism for lists are defined by

$$
\begin{aligned}
&\mathit{inn}\ (\mathit{Left}\ \ ())\ \ \ \ \ \ = [\,]\\
&\mathit{inn}\ (\mathit{Right}\ (x, xs)) = x : xs\\
&\mathit{out}\ [\,]\ \ \ \ \ \ = \mathit{Left}\ \ ()\\
&\mathit{out}\ (x : xs) = \mathit{Right}\ (x, xs)
\end{aligned}
$$

How do we define pattern matching? As we mentioned above, the result of matching a value against a pattern is an environment indexed by the variable positions of the pattern. For example, matching a list against the cons pattern

$$\mathit{PIn}\ (\mathit{PRight}\ (\mathit{PProd}\ \mathit{PVar}\ \mathit{PVar})) \tag{3}$$

should produce an environment containing its head and tail. Here we want a safe (but not necessarily efficient) representation of the environment type, in the sense

that the indices into the environment should be exactly the variable positions of the pattern, and we want that to be enforced statically by typechecking. In other words, this environment type depends on the pattern, and a way to compute this type is to encode it as a second index of the *Pat* datatype:

```
data Pat a env where
  PVar   :: Eq a ⇒ Pat a (Var a)
  PConst :: Eq a ⇒ a → Pat a ()
  PProd  :: Pat a a' → Pat b b' b'' → Pat (a, b) (a', b')
  PLeft  :: Pat a a' → Pat (Either a b) a'
  PRight :: Pat b b' → Pat (Either a b) b'
  PIn    :: InOut a ⇒ Pat (F a) b → Pat a b
```

Notice that an environment type is just a product of *Var* types — for example, the environment type computed for the cons pattern (3) is

$$(Var\ a, Var\ [a]) \tag{4}$$

We will discuss *Var* later, which is simply defined by

```
newtype Var a = Var a
```

Now we can define the (strongly typed) pattern matching operation:

```
deconstruct :: Pat a env → a → Maybe env
deconstruct PVar          x          = return (Var x)
deconstruct (PConst c)    x          = if c ≡ x then return () else Nothing
deconstruct (l `PProd` r) (x, y)     = liftM2 (,) (deconstruct l x)
                                                  (deconstruct r y)
deconstruct (PLeft p)     (Left x)   = deconstruct p x
deconstruct (PLeft _)     _          = Nothing
deconstruct (PRight p)    (Right x)  = deconstruct p x
deconstruct (PRight _)    _          = Nothing
deconstruct (PIn p)       x          = deconstruct p (out x)
```

and its inverse (which is total):

```
construct :: Pat a env → env → a
construct PVar          (Var x)      = x
construct (PConst c)    _            = c
construct (l `PProd` r) (envl, envr) = (construct l envl, construct r envr)
construct (PLeft p)     env          = Left  (construct p env)
construct (PRight p)    env          = Right (construct p env)
construct (PIn p)       env          = inn (construct p env)
```

Precisely speaking, we have

$$deconstruct\ p\ x = Just\ e \quad \Leftrightarrow \quad construct\ p\ e = x$$

for all $p :: Pat\ a\ env$, $x :: a$, and $e :: env$, establishing a (half-) partial isomorphism between env and a.

Now consider view rearrangement, which evaluates a "simple" pattern-matching λ-expression on the view and continues execution with the transformed view. The body of the λ-expression refers to the variables appearing in the pattern. How do we represent such references? We have seen that an environment type is a product, i.e., a binary tree; to refer to a component in an environment, we can use a *path* that goes from the root to a sub-tree. In BiGUL, these paths are called *directions*:

```
data Direction env a where
  DVar   :: Direction (Var a) a
  DLeft  :: Direction a t → Direction (a, b) t
  DRight :: Direction b t → Direction (a, b) t
```

The type of a direction is indexed by the environment type it points into and the component type it points to. Note that the type of *DVar* is specified to work with only environment types marked with *Var*; this is for ensuring that a direction goes all the way down to an actual component at a variable position of the pattern, rather than stopping half-way and pointing to a sub-tree which include more than one component. For example, for the environment type (4) for the cons pattern, only two directions are valid, namely *DLeft DVar* and *DRight DVar*, whereas *DVar* alone would point to the entire environment instead of one of the variable positions, and is ruled out by typechecking (in the sense that it is impossible for *DVar* to have type *Direction* (*Var a*, *Var* [*a*]) *b* for any *b*). It is easy to extract a component from an environment following a direction:

```
retrieve :: Direction env a → env → a
retrieve DVar       (Var x) = x
retrieve (DLeft d)  (x, _)  = retrieve d x
retrieve (DRight d) (_, y)  = retrieve d y
```

Now we can define *expressions*, which are similar to patterns but include directions rather than variables, to represent the body of rearranging λ-expressions:

```
data Expr env a where
  EDir   :: Direction env a → Expr env a
  EConst :: (Eq a) ⇒ a → Expr env a
  EProd  :: Expr env a → Expr env b → Expr env (a, b)
  ELeft  :: Expr env a → Expr env (Either a b)
  ERight :: Expr env b → Expr env (Either a b)
  EIn    :: (InOut a) ⇒ Expr env (F a) → Expr env a
```

For example, the rearranging λ-expression

$$\lambda(x : xs) \to (x, xs) \tag{5}$$

is represented by the cons pattern (3) and the pair expression

$$EProd\ (EDir\ (DLeft\ DVar))\ (EDir\ (DRight\ DVar)) \tag{6}$$

Evaluating an expression under an environment is similar to inverse pattern matching:

$$
\begin{array}{ll}
\mathit{eval} :: \mathit{Expr}\ \mathit{env}\ a \to \mathit{env} \to a & \\
\mathit{eval}\ (\mathit{EDir}\ d) & \mathit{env} = \mathit{retrieve}\ d\ \mathit{env} \\
\mathit{eval}\ (\mathit{EConst}\ c) & \mathit{env} = c \\
\mathit{eval}\ (l\ \text{`}\mathit{EProd}\text{`}\ r) & \mathit{env} = (\mathit{eval}\ l\ \mathit{env}, \mathit{eval}\ r\ \mathit{env}) \\
\mathit{eval}\ (\mathit{ELeft}\ e) & \mathit{env} = \mathit{Left}\ \ (\mathit{eval}\ e\ \mathit{env}) \\
\mathit{eval}\ (\mathit{ERight}\ e) & \mathit{env} = \mathit{Right}\ (\mathit{eval}\ e\ \mathit{env}) \\
\mathit{eval}\ (\mathit{EIn}\ e) & \mathit{env} = \mathit{inn}\ (\mathit{eval}\ e\ \mathit{env})
\end{array}
$$

The type of RearrV is then:

$$\mathit{RearrV} :: \mathit{Pat}\ v\ \mathit{env} \to \mathit{Expr}\ \mathit{env}\ v' \to \mathit{BiGUL}\ s\ v' \to \mathit{BiGUL}\ s\ v$$

Note that in the type of RearrV, the types of the pattern and expression share the same environment type index, ensuring that the directions in the expression can only refer to the variable positions in the pattern. And the *put* behavior of RearrV is simply:

$$
\begin{array}{l}
\mathit{put}\ (\mathit{RearrV}\ p\ e\ b)\ s\ v = \mathbf{do}\ \mathit{env} \leftarrow \mathit{deconstruct}\ p\ v \\
\qquad\qquad\qquad\qquad\qquad\qquad\quad \mathit{put}\ b\ s\ (\mathit{eval}\ e\ \mathit{env})
\end{array}
$$

For the *get* direction, after executing the inner BiGUL program to obtain an intermediate view, we should reverse the roles of the pattern and body in the rearranging λ-expression $\lambda p \to e$, using e as a (possibly non-linear) pattern to match the intermediate view, and computing the final view by evaluating p. For example, the *put* direction of view rearrangement with the λ-expression (5) turns a view list into a pair, on which the inner program operates; in the *get* direction, the inner program will extract from the source an intermediate view pair, which should be converted back to a list by the inverse λ-expression $\lambda(x, xs) \to (x : xs)$. In more detail, given an intermediate view pair (x, xs), we match it with the pair expression (6), and see that x is associated with the direction $\mathit{DLeft}\ \mathit{DVar}$ and xs with $\mathit{DRight}\ \mathit{DVar}$. From such associations we can reconstruct an environment of type (4) with x and xs in the right places, and then we can evaluate the cons pattern (3) in this reconstructed environment, arriving at the final view $x : xs$.

In general, the intermediate view will be decomposed according to the body expression, and eventually each of its components will be paired with a direction indicating which variable position the component should go into in the reconstructed environment. To do the reconstruction, we can prepare a "container" which is similar to an environment except that the variable positions are initially empty. For each pair of a component and a direction, we try to put that component into the place in the container pointed to by the direction; if two components are put into the same position twice (indicating that the λ-expression uses a variable more than once), then they must be equal. In the end, we check that all places in the container are filled, and then use it as an environment to evaluate the pattern. Again, to compute the type of containers from a pattern, we add a third index to *Pat*:

```
data Pat a env con where
  PVar   :: Eq a ⇒ Pat a (Var a) (Maybe a)
  PConst :: Eq a ⇒ a → Pat a () ()
  PProd  :: Pat a a' a'' → Pat b b' b'' → Pat (a, b) (a', b') (a'', b'')
  PLeft  :: Pat a a' a'' → Pat (Either a b) a' a''
  PRight :: Pat b b' b'' → Pat (Either a b) b' b''
  PIn    :: InOut a ⇒ Pat (F a) b c → Pat a b c
```

A container type is just like an environment type except that the variable positions give rise to *Maybe* instead of *Var*. For the cons example, the computed container type is

$$(Maybe\ a, Maybe\ [a]) \tag{7}$$

The first step — matching a value with an expression — can then be implemented as:

```
uneval :: Pat a env con → Expr env b → b → con → Maybe con
uneval p (EDir d)    x          con = unevalD p d x con
uneval p (EConst c)  x          con = if c ≡ x then return con
                                      else Nothing
uneval p (EProd l r) (x, y)     con = uneval p l x con >>= uneval p r y
uneval p (ELeft e)   (Left x)   con = uneval p e x con
uneval p (ELeft _)   x          con = Nothing
uneval p (ERight e)  (Right x)  con = uneval p e x con
uneval p (ERight _)  x          con = Nothing
uneval p (EIn e)     x          con = uneval p e (out x) con

unevalD :: Pat a env con → Direction env b → b → con → Maybe con
unevalD PVar         DVar       x (Just y)      = if x ≡ y
                                                  then return (Just x)
                                                  else Nothing
unevalD PVar         DVar       x Nothing       = return (Just x)
unevalD (PConst c)   _          x con           = return con
unevalD (l `PProd` r) (DLeft  d) x (conl, conr) = liftM (, conr)
                                                    (unevalD l d x conl)
unevalD (l `PProd` r) (DRight d) x (conl, conr) = liftM (conl, )
                                                    (unevalD r d x conr)
unevalD (PLeft p)    d          x con           = unevalD p d x con
unevalD (PRight p)   d          x con           = unevalD p d x con
unevalD (PIn p)      d          x con           = unevalD p d x con
```

This function *uneval* initially takes an empty container, which is generated by:

```
emptyContainer :: Pat v env con → con
emptyContainer PVar          = Nothing
emptyContainer (PConst c)    = ()
emptyContainer (l `PProd` r) = (emptyContainer l, emptyContainer r)
emptyContainer (PLeft p)     = emptyContainer p
```

$$
\begin{aligned}
&\mathit{emptyContainer}\ (\mathit{PRight}\ p) &&= \mathit{emptyContainer}\ p\\
&\mathit{emptyContainer}\ (\mathit{PIn}\ p) &&= \mathit{emptyContainer}\ p
\end{aligned}
$$

And then we can try to convert a container to an environment, checking whether the container is full in the process:

$$
\begin{aligned}
&\mathit{fromContainerV} :: \mathit{Pat}\ v\ \mathit{env}\ \mathit{con} \to \mathit{con} \to \mathit{Maybe}\ \mathit{env}\\
&\mathit{fromContainerV}\ \mathit{PVar} && \mathit{Nothing} && = \mathit{Nothing}\\
&\mathit{fromContainerV}\ \mathit{PVar} && (\mathit{Just}\ v) && = \mathit{return}\ (\mathit{Var}\ v)\\
&\mathit{fromContainerV}\ (\mathit{PConst}\ c) && \mathit{con} && = \mathit{return}\ ()\\
&\mathit{fromContainerV}\ (l\ \text{`}\mathit{PProd}\text{`}\ r) && (\mathit{conl}, \mathit{conr}) && = \mathit{liftM2}\ (,)\\
&&&&&\quad (\mathit{fromContainerV}\ l\ \ \mathit{conl})\\
&&&&&\quad (\mathit{fromContainerV}\ r\ \mathit{conr})\\
&\mathit{fromContainerV}\ (\mathit{PLeft}\ p) && \mathit{con} && = \mathit{fromContainerV}\ \mathit{pat}\ \mathit{con}\\
&\mathit{fromContainerV}\ (\mathit{PRight}\ p) && \mathit{con} && = \mathit{fromContainerV}\ \mathit{pat}\ \mathit{con}\\
&\mathit{fromContainerV}\ (\mathit{PIn}\ p) && \mathit{con} && = \mathit{fromContainerV}\ \mathit{pat}\ \mathit{con}
\end{aligned}
$$

We can let out a sigh of relief once we successfully get hold of an environment, since the last step — inverse pattern matching — is total. To sum up:

$$
\begin{aligned}
\mathit{get}\ (\mathit{RearrV}\ p\ e\ b)\ s = \mathbf{do}\ &v' \leftarrow \mathit{get}\ b\ s\\
&\mathit{con} \leftarrow \mathit{uneval}\ p\ e\ v'\ (\mathit{emptyContainer}\ p)\\
&\mathit{env} \leftarrow \mathit{fromContainerV}\ p\ \mathit{con}\\
&\mathit{return}\ (\mathit{construct}\ p\ \mathit{env})
\end{aligned}
$$

To be concrete, let us go through the steps of inverse rearranging in the cons example. Starting from an intermediate view (x, xs) and an empty container $(\mathit{Nothing}, \mathit{Nothing})$ of type (7), *uneval* will invoke *unevalD* twice, the first time updating the container to $(\mathit{Just}\ x, \mathit{Nothing})$ and the second time to $(\mathit{Just}\ x, \mathit{Just}\ xs)$. The resulting container is full, and thus *fromContainerV* will successfully turn it into an environment $(\mathit{Var}\ x, \mathit{Var}\ xs)$ of type (4), in which we evaluate the cons pattern (3) and obtain $x : xs$.

Conceptually, this is just reversing pattern matching and expression evaluation. To actually prove the well-behavedness, though, we need to reason about stateful computation (which is what *uneval* essentially is), which involves coming up with suitable invariants and proving that they are maintained throughout the computation.

It is interesting to mention that there would be a catch if we designed this combinator from the *get* direction: It is tempting to think that, since a rearranging λ-expression gives rise to a partial isomorphism, which can be lifted to a lens, we can simply compose the lens lifted from the isomorphism with the inner lens to give a lens semantics to RearrV. This would result in a redundant computation of an intermediate source which is immediately discarded, and now the success of the whole computation would unnecessarily depend on that of the intermediate source. To eliminate the redundant computation, we would need to use a special composition which composes a lens directly with an isomorphism on the right. Such a need would be hard to notice since the *get* behavior of the

two compositions are the same; that is, we really have to think in terms of *put* to see that the special composition is needed.

6 Position-based, key-based, and delta-based list alignment

In the next three sections, we will talk about some applications in BiGUL, starting from the list alignment problem. List alignment is one of the tasks that frequently show up when developing bidirectional applications. When the source and view are both lists, and the *get* direction (i.e., the consistency relation) is a *map*, how do we put an updated view — the updates on which might involve insertions, deletions, in-place modifications, and reordering — into the source?

Throughout the section, we use a concrete example to introduce three variations of list alignment. Suppose that we represent a payroll database as a list. (This is a slightly inadequate setting for explaining list alignment, because entries in a database are usually unordered. But let us assume that order matters.) Each entry is a triple consisting of an identification number ("id" henceforth), a name, and a salary number:

$$\begin{aligned}&\textbf{type } \mathit{Source} = (\mathit{Id}, (\mathit{Name}, \mathit{Salary}))\\ &\textbf{type } \mathit{Id} \quad\ = \mathit{Int}\\ &\textbf{type } \mathit{Name} \ = \mathit{String}\\ &\textbf{type } \mathit{Salary} = \mathit{Int}\end{aligned}$$

For example, here is a sample payroll database:

$$\begin{aligned}&\mathit{employees} :: [\mathit{Source}]\\ &\mathit{employees} = [\,(0, (\texttt{"Zhenjiang"}, 1000))\\ &\qquad\qquad\ \ ,(1, (\texttt{"Josh"}\ \ \ \ \ \ , 400\ \ ))\\ &\qquad\qquad\ \ ,(2, (\texttt{"Jeremy"}\ \ \ , 2000))]\end{aligned}$$

Suppose that the human resource department is in charge of hiring or sacking employees but has no say on salary numbers, so the entries of the database are presented to them only as pairs of ids and names:

$$\textbf{type } \mathit{View} = (\mathit{Id}, \mathit{Name})$$

For example, *employees* is presented to them as

$$[(0, \texttt{"Zhenjiang"}), (1, \texttt{"Josh"}), (2, \texttt{"Jeremy"})]$$

on which they can make modifications. It is easy to write a BiGUL program to synchronize the source and view elements:

$$\begin{aligned}&\mathit{bx} :: \mathit{BiGUL\ Source\ View}\\ &\mathit{bx} = {}^{\$}(\mathit{rearrV}\ [\![\, \lambda(\mathit{id}, \mathit{name}) \to (\mathit{id}, (\mathit{name}, ()))\,]\!]^{\$}\\ &\qquad\quad \mathit{Replace}\ \text{`}\mathit{Prod}\text{`}\ (\mathit{Replace}\ \text{`}\mathit{Prod}\text{`}\ \mathit{Skip}\ (\mathit{const}\ ()))\end{aligned}$$

The problem is then how the correspondences between sources and views in the two lists can be determined, so that *bx* can be applied to the right pairs.

6.1 Position-based alignment

As a first exercise, we consider the simplest strategy, which matches source and view elements by their positions in the lists. If the source list has more elements than the view list, the extra elements at the tail are simply dropped; if the source list has less elements, then new source elements have to be created, which we can specify as a function:

$$
\begin{aligned}
&cr :: View \to Source \\
&cr\ (i, n) = (i, (n, 0))
\end{aligned}
$$

The salary is set to zero, which can be taken care of by the accounting department later, for example. In general, we can abstract the source and view types and *bx* and *cr* as parameters, so the alignment program we write can be widely applicable. Here is how we implement position-based alignment, which is fairly standard:

$$
\begin{aligned}
&posAlign :: (Show\ s, Show\ v) \Rightarrow BiGUL\ s\ v \to (v \to s) \to BiGUL\ [s]\ [v] \\
&posAlign\ b\ c = Cas \\
&\quad [\,^{\$}(normalSV\ {}^{\mathbb{P}}[\![\,]\!]\ {}^{\mathbb{P}}[\![\,]\!]\ {}^{\mathbb{P}}[\![\,]\!]) \\
&\qquad \Longrightarrow {}^{\$}(update\ {}^{\mathbb{P}}[\![\,]\!]\ {}^{\mathbb{P}}[\![\,]\!]\ {}^{\mathbb{D}}[\![\,]\!]) \\
&\quad ,\,^{\$}(normalSV\ {}^{\mathbb{P}}[\![\,\_ : \_\,]\!]\ {}^{\mathbb{P}}[\![\,\_ : \_\,]\!]\ {}^{\mathbb{P}}[\![\,\_ : \_\,]\!]) \\
&\qquad \Longrightarrow {}^{\$}(update\ {}^{\mathbb{P}}[\![\,x : xs\,]\!]\ {}^{\mathbb{P}}[\![\,x : xs\,]\!]\ {}^{\mathbb{D}}[\![\,x = b; xs = posAlign\ b\ c\,]\!]) \\
&\quad ,\,^{\$}(adaptiveSV\ {}^{\mathbb{P}}[\![\,\_ : \_\,]\!]\ {}^{\mathbb{P}}[\![\,]\!]) \\
&\qquad \Longrightarrow \lambda\_\ \_ \to [\,] \\
&\quad ,\,^{\$}(adaptiveSV\ {}^{\mathbb{P}}[\![\,]\!]\ {}^{\mathbb{P}}[\![\,\_ : \_\,]\!]) \\
&\qquad \Longrightarrow \lambda\_\ (v : \_) \to [\,c\ v\,] \\
&\quad]
\end{aligned}
$$

The normal branches deal with the situations where both lists are empty or non-empty, and the adaptive branches remove or create elements when the lengths of the two lists differ.

The *get* direction of *posAlign* does exactly what we want it to do:

```
*> get (posAlign bx cr) employees
Just [(0,"Zhenjiang"),(1,"Josh"),(2,"Jeremy")]
```

It should be quite obvious, though, that the *put* direction is not so useful for our purpose. If we sack Josh:

$$
\begin{aligned}
&updatedEmployees0 :: [View] \\
&updatedEmployees0 = [(0, \texttt{"Zhenjiang"}), (2, \texttt{"Jeremy"})]
\end{aligned}
$$

then the database will updated to:

```
*> put (posAlign bx cr) employees updatedEmployees0
Just [(0,("Zhenjiang",1000)),(2,("Jeremy",400))]
```

where Jeremy inadvertently gets Josh's original salary. Even if we do not remove any employee, we may still want to reorder them:

$$\begin{aligned}&\mathit{updatedEmployees1} :: [\mathit{View}]\\&\mathit{updatedEmployees1} = [(2, \texttt{"Jeremy"}), (0, \texttt{"Zhenjiang"}), (1, \texttt{"Josh"})]\end{aligned}$$

and now everyone gets a wrong salary:

```
*> put (posAlign bx cr) employees updatedEmployees1
Just [(2,("Jeremy",1000)),(0,("Zhenjiang",400)),(1,("Josh",2000))]
```

This first exercise shows that the alignment problem is inherently one that should be solved from the *put* direction. It is easy to implement the *get* direction correctly, but what matters is the *put* behavior.

6.2 Key-based alignment

A more reasonable strategy is to match source and view elements by some *key* value. In our example, we can use the id as the key. Key-based alignment might seem much more complex than position-based alignment, but, in fact, we can just revise *posAlign* to get a BiGUL program for key-based alignment!

First of all, we need to somehow obtain the keys. In our example, on both the source and view we can use *fst* to extract the key value. In general, we can further parametrize the alignment program with key extraction functions:

$$\begin{aligned}&\mathit{keyAlign} :: \mathit{forall}\ a\ b\ k \circ (\mathit{Show}\ a, \mathit{Show}\ b, \mathit{Eq}\ k)\\&\qquad \Rightarrow (a \to k) \to (b \to k) \to \mathit{BiGUL}\ a\ b \to (b \to a) \to \mathit{BiGUL}\ [a]\ [b]\end{aligned}$$

The first normal branch of *posAlign* still works perfectly. As for the second normal branch, we should revise the main condition to also require that the head elements of the two lists have the same key value:

$$\lambda(s : ss)\ (v : vs) \to \mathit{ks}\ s \equiv \mathit{kv}\ v$$

The first adaptive branch, again, works well. The second adaptive branch, on the other hand, is no longer applicable: Since the main condition of the second normal branch has been tightened, it is no longer the case that this adaptive branch will receive only empty source lists. In fact, whether the source list is empty or not is not relevant here — what matters now is whether the key of the first view is in the source list. If it is, then we bring the (first) source element with the same key value to the head position, and the second normal branch can take over; otherwise, we create a new source element. This gives us key-based alignment:

$$\begin{aligned}&\mathit{keyAlign} :: \mathit{forall}\ s\ v\ k \circ (\mathit{Show}\ s, \mathit{Show}\ v, \mathit{Eq}\ k)\\&\qquad \Rightarrow (s \to k) \to (v \to k) \to \mathit{BiGUL}\ s\ v \to (v \to s) \to \mathit{BiGUL}\ [s]\ [v]\\&\mathit{keyAlign}\ \mathit{ks}\ \mathit{kv}\ b\ c = \mathit{Case}\\&\quad [\$(\mathit{normalSV}\ {}^{\mathbb{P}}[\![\,]\!]\ {}^{\mathbb{P}}[\![\,]\!]\ {}^{\mathbb{P}}[\![\,]\!])\end{aligned}$$

```
    ⟹ $(update ℙ⟦[]⟧ ℙ⟦[]⟧ 𝔻⟦⟧)
  , $(normal ⟦λ(s : ss) (v : vs) → ks s ≡ kv v⟧ ℙ⟦_ : _⟧)
    ⟹ $(update ℙ⟦x : xs⟧ ℙ⟦x : xs⟧
                          𝔻⟦x = b; xs = keyAlign ks kv b c⟧)
  , $(adaptiveSV ℙ⟦_ : _⟧ ℙ⟦[]⟧)
    ⟹ λ_ _ → []
  , $(adaptive ⟦λss (v : vs) → kv v ∈ map ks ss⟧)
    ⟹ λss (v : _) → uncurry (:) (extract (kv v) ss)
  , $(adaptiveSV ℙ⟦_⟧ ℙ⟦_ : _⟧)
    ⟹ λss (v : _) → c v : ss
  ]
  where
    extract :: k → [s] → (s, [s])
    extract k (x : xs) | ks x ≡ k  = (x, xs)
                       | otherwise = let (y, ys) = extract k xs
                                     in  (y, x : ys)
```

Back to our payroll database example. The *get* direction behaves the same:

```
*> get (keyAlign fst fst bx cr) employees
Just [(0,"Zhenjiang"),(1,"Josh"),(2,"Jeremy")]
```

Unlike position-based alignment, view element deletion can now be reflected correctly:

```
*> put (keyAlign fst fst bx cr) employees updatedEmployees0
Just [(0,("Zhenjiang",1000)),(2,("Jeremy",2000))]
```

And reordering as well:

```
*> put (keyAlign fst fst bx cr) employees updatedEmployees1
Just [(2,("Jeremy",2000)),(0,("Zhenjiang",1000)),(1,("Josh",400))]
```

So it seems that key-based alignment is just what we need. Indeed, key-based alignment usually works well, but there is an important assumption: The key values should not be changed. If, for example, we decide to assign a different id to Josh:

```
updatedEmployees2 :: [View]
updatedEmployees2 = [(0, "Zhenjiang"), (100, "Josh"), (1, "Jeremy")]
```

Then the effect is the same as sacking Josh and then hiring him again, whose salary is thus reset:

```
*> put (keyAlign fst fst bx cr) employees updatedEmployees2
Just [(0,("Zhenjiang",1000)),(100,("Josh",0)),(1,("Jeremy",400))]
```

The problem is that we cannot distinguish modification from deletion and insertion pairs. To be able to have such distinction, we introduce the notion of *deltas*, with which the links between source and view elements can be kept track of.

6.3 Delta-based alignment

A (horizontal) *delta* between a source list and a view list is a list of pairs of associated positions:

$$\textbf{type}\ \mathit{Delta} = [(\mathit{Int}, \mathit{Int})]$$

For example, the delta we have in mind between the source list *employees* and the view list *updatedEmployees2* is $[(0,0),(1,1),(2,2)]$, which, in particular, associates the source and view entries for Josh since $(2,2)$ is included, instead of $[(0,0),(1,1)]$, which indicates that Josh's source entry is not associated with any view entry and should be deleted, and that Josh's view entry is not associated with any source entry and is thus new. Deltas can easily represent reordering as well. For example, we would supply the delta between *employees* and *updatedEmployees1* as $[(0,1),(1,2),(2,0)]$, associating the 0th element in the source — namely the one for Zhenjiang — with the 1st element in the view, and so on.

So the input now includes not only source and view lists but also a delta between them. Recall key-based alignment: What it does overall is to bring the first matching source element to the front for each view element, so the source list is updated throughout execution, with the links between the source and view elements gradually and implicitly restored. If we are doing something similar with delta-based alignment, then when the source list is updated, the delta should also be updated to reflect the restored consistency. This suggests that the delta should be paired with the source list, so it can be updated. The type we use for the delta-based alignment program is thus:

$$\begin{aligned}&\mathit{deltaAlign}\ ::\ (\mathit{Show}\ s, \mathit{Show}\ v)\\ &\qquad\Rightarrow \mathit{BiGUL}\ s\ v \to (v \to s) \to \mathit{BiGUL}\ ([s], \mathit{Delta})\ [v]\end{aligned}$$

Here we take a simpler approach to implementing *deltaAlign*, analyzing the problem into just two cases: The delta can tell us either that the source and view elements are all in correspondence, in which case a simple position-based alignment suffices, or that we need to do some rearrangement of the source elements, which can be done by adaptation. In BiGUL:

$$\begin{aligned}&\mathit{idDelta}\ ::\ [s] \to \mathit{Delta}\\ &\mathit{idDelta}\ ss\ =\ [(i,i)\mid i \leftarrow [0\,..\,\mathit{length}\ ss]]\\ &\mathit{deltaAlign}\ ::\ (\mathit{Show}\ s, \mathit{Show}\ v)\\ &\qquad\Rightarrow \mathit{BiGUL}\ s\ v \to (v \to s) \to \mathit{BiGUL}\ ([s], \mathit{Delta})\ [v]\\ &\mathit{deltaAlign}\ b\ c = \mathit{Case}\\ &\quad[\,^{\$}(\mathit{normal}\ [\![\,\lambda(ss,d)\ vs \to \mathit{length}\ ss \equiv \mathit{length}\ vs \wedge d \equiv \mathit{idDelta}\ ss\,]\!]\\ &\qquad\qquad{}^{\mathbb{P}}[\![\,\_\,]\!])\\ &\qquad\Longrightarrow {}^{\$}(\mathit{rearrV}\ [\![\,\lambda vs \to (vs,())\,]\!])^{\$}\mathit{posAlign}\ b\ c\ \text{`}\mathit{Prod}\text{`}\ \mathit{Skip}\ (\mathit{const}\ ())\\ &\quad,{}^{\$}(\mathit{adaptive}\ [\![\,\lambda\_\ \_ \to \mathit{otherwise}\,]\!])\\ &\qquad\Longrightarrow \lambda(ss,d)\ vs \to\\ &\qquad\qquad\textbf{let}\ d'\ = \mathit{map}\ \mathit{swap}\ d\end{aligned}$$

$$ss' = [maybe\ (c\ v)\ (ss!!)\ (lookup\ j\ d')\ |\ (v, j) \leftarrow zip\ vs\ [0\,..]]$$
$$\textbf{in}\ (ss', idDelta\ ss')$$
$$]$$

The source and view lists are in full correspondence if and only if they have the same length and the delta associates all their elements positionally. This full positional delta can be computed by *idDelta*. When this is the case, it suffices to call *posAlign* to carry out element-wise synchronization, since no rearrangement is required. Otherwise, we enter the adaptive branch, which constructs a new source list in full correspondence with the view list, drawing elements from the original source list or create new ones as the delta dictates. The new source list is in full correspondence with the view list, so the delta we pair with it is the one computed by *idDelta*.

Only when performing *put* does a delta make sense. When performing *get*, however, we still need to supply a delta since it is part of the source; but there is a natural choice, namely *idDelta*. So we define:

$$putDeltaAlign\ ::\ (Show\ s, Show\ v) \Rightarrow BiGUL\ s\ v \to (v \to s) \to [s] \to Delta \to [v] \to Maybe\ [s]$$
$$putDeltaAlign\ b\ c\ ss\ d\ vs = fmap\ fst\ (put\ (deltaAlign\ b\ c)\ (ss, d)\ vs)$$
$$getDeltaAlign\ ::\ (Show\ s, Show\ v) \Rightarrow BiGUL\ s\ v \to (v \to s) \to [s] \to Maybe\ [v]$$
$$getDeltaAlign\ b\ c\ ss = get\ (deltaAlign\ b\ c)\ (ss, idDelta\ ss)$$

It is easy to prove that, given the same b and c, these two functions do form a lens. The key observation is that the delta produced by *put* (*deltaAlign* b c) is necessarily one computed by *idDelta*, so, for example, in PutGet, throwing away the delta in the *put* direction is fine because it can be recomputed by *idDelta*, and the *get* direction can resume from exactly the same source pair.

Back to our example. We can now update Josh's id without resetting his salary by providing a full delta indicating that there are only in-place updates:

```
*> putDeltaAlign bx cr employees
     [(0,0), (1,1), (2,2)] updatedEmployees2
Just [(0,("Zhenjiang",1000)),(100,("Josh",400)),(1,("Jeremy",2000))]
```

Besides obvious modifications like reordering, we can also do some fairly subtle modifications now: If we actually sack Josh and replace him with a new Josh (inheriting the original Josh's id) whose salary should be reset (to be re-considered by the accounting department), we can say so by providing a partial delta:

```
*> putDeltaAlign bx cr employees [(0,0), (2,2)]
     =<< getDeltaAlign bx cr employees
Just [(0,("Zhenjiang",1000)),(1,("Josh",0)),(2,("Jeremy",2000))]
```

One alignment to rule them all. Where do deltas come from? In general, we may provide a special view editor which monitors how the view is modified and

produces a suitable delta. But in more specialized scenarios, deltas can simply be computed by, for example, comparing the source and view. We can formalize this delta computation as:

$$\textbf{type}\ \mathit{DeltaStrategy}\ s\ v = [s] \to [v] \to \mathit{Delta}$$

and further parametrize *putDeltaAlign*:

$$\begin{aligned}&\mathit{putDeltaAlignS}\ ::\ (\mathit{Show}\ s, \mathit{Show}\ v) \Rightarrow \mathit{DeltaStrategy}\ s\ v\\ &\qquad\qquad \to \mathit{BiGUL}\ s\ v \to (v \to s) \to [s] \to [v] \to \mathit{Maybe}\ [s]\\ &\mathit{putDeltaAlignS}\ \mathit{dst}\ b\ c\ \mathit{ss}\ \mathit{vs} = \mathit{putDeltaAlign}\ b\ c\ \mathit{ss}\ (\mathit{dst}\ \mathit{ss}\ \mathit{vs})\ \mathit{vs}\end{aligned}$$

Position-based and key-based alignment can then be seen as special cases of delta-based alignment using specific delta-computing strategies. For position-based alignment, we simply compute the identity delta:

$$\begin{aligned}&\mathit{byPosition} :: \mathit{DeltaStrategy}\ s\ v\\ &\mathit{byPosition}\ \mathit{ss}\ \_ = \mathit{idDelta}\ \mathit{ss}\end{aligned}$$

And for key-based alignment, we compute a delta associating source and view elements with the same key:

$$\begin{aligned}&\mathit{byKey} :: \mathit{Eq}\ k \Rightarrow (s \to k) \to (v \to k) \to \mathit{DeltaStrategy}\ s\ v\\ &\mathit{byKey}\ \mathit{ks}\ \mathit{kv}\ \mathit{ss}\ \mathit{vs} =\\ &\quad \textbf{let}\ \mathit{sis} = \mathit{zip}\ \mathit{ss}\ [0\,..]\\ &\quad \textbf{in}\ \ \mathit{catMaybes}\ [\ \mathit{fmap}\ (\lambda(\_, i) \to (i, j))\ (\mathit{find}\ (\lambda(s, \_) \to \mathit{ks}\ s \equiv \mathit{kv}\ v)\ \mathit{sis})\\ &\qquad\qquad\qquad\quad \mid (v, j) \leftarrow \mathit{zip}\ \mathit{vs}\ [0\,..]]\end{aligned}$$

We can check that these strategies indeed give us position-based and key-based alignment:

```
*> putDeltaAlignS byPosition bx cr employees updatedEmployees0
Just [(0,("Zhenjiang",1000)),(2,("Jeremy",400))]

*> putDeltaAlignS (byKey fst fst) bx cr employees updatedEmployees1
Just [(2,("Jeremy",2000)),(0,("Zhenjiang",1000)),(1,("Josh",400))]
```

7 Bidirectionalizing relational queries with BiGUL

In work on relational databases, the view-update problem is about how to translate update operations on the view table to corresponding update operations on the source table properly. Relational lenses [3] try to solve this problem by providing a list of combinators that let the user write get functions (queries) with specified updated policies for put functions (updates); however this can only provide limited control of update policies. To resolve this problem, we define a new library BRUL [16], where two *putback*-based combinators (operators) are designed to specify update policies, from which forward queries (selection, projection, join) can be automatically derived.

Track	Date	Rating	Album	Quantity
Lullaby	1989	3	Galore	2
Lullaby	1989	3	Show	3
Lovesong	1989	5	Galore	2
Lovesong	1989	5	Paris	4
Trust	1992	4	Wish	5

Fig. 1. Source table

- *align* is to update a source list with a view list by aligning part of source elements filtered by a predicate with view elements according to a matching criteria between source element and view element;
- *unjoin* is to decompose a join view to update two sources.

In this tutorial, we will focus on *align*, which can describe flexible update strategies related to selection/projection queries.

7.1 Relational database representation

A relational table is a list of records, and each record is a list of attribute elements of type.

```
type RT = [Record]
type Record = [RType]
data RType = RInt Int
           | RString String
           | RFloat Float
           | RDouble Double
  deriving (Show, Eq, Ord)
```

Here we introduce a new datatype *RType*. To do pattern matching on the data of this type, we need to declare

```
deriveBiGULGeneric '' RType
```

As an example, consider the table in Figure 1 that stores five music track records, and each record contains its Track name, release Date, Rating, Album, and the Quantity of this Album. It can be represented as follows.

```
s = [[RString "Lullaby", RInt 1989, RInt 3, RString "Galore", RInt 1]
    ,[RString "Lullaby", RInt 1989, RInt 3, RString "Show", RInt 3]
    ,[RString "Lovesong", RInt 1989, RInt 5, RString "Galore", RInt 1]
    ,[RString "Lovesong", RInt 1989, RInt 5, RString "Disintegration", RInt 4]
    ,[RString "Trust",    RInt 1992, RInt 4, RString "Wish", RInt 5]
    ]
```

A table may have functional dependencies. We use *FDMap* to store functional dependencies of a table

type *FDMap* = *Map.Map Int* [*Int*]

It maps from one attribute to a list of attributes that depend on it. Here each attribute is represented by the index in the record list.

Return to the above example. There are functional dependencies from Track to Date (denoted as $Track \rightarrow Date$), Track to Rating (denoted as $Track \rightarrow Rating$), and Album to Quantity (denoted as $Album \rightarrow Quantity$). This dependencies can be specified by

sfdMap :: *FDMap*
sfdMap = *Map.fromList* $[(0, [1, 2]), (3, [4])]$

7.2 Relation Alignment

The alignment of two relational tables is similar to the key-based list alignment in Section 6. The difference is that we need to consider filtering on the source records and maintaining of functional dependency on the source elements when updates on the view happen.

Our relation alignment has the form of

relAlign p ks kv b c h fd

where p is a predicate for filtering out those source elements that do not satisfy p, ks and kv are two functions to extract keys from the source and the view respectively, b is a BiGUL program to do updating when the source and the view are matched by their keys, c is a function for creating a source element, h is a function to conceal elements in the source, and fd is a function for updating source records according to the functional dependency. Concretely, *relAlign* uses p to extract the satisfied source records, and then uses ks and kv to match these source elements with the view elements. The matching result has three cases, and each case uses different update operation: when source and view elements are matched, b is used to update the source element by the view element; when a view element has no corresponding matching source element, c is used to create a source element from this view element; when a source element has no corresponding matching view element, h is used to conceal the element (from the view) by either deleting this source element or modifying it so that it does not satisfy the filter condition.

Let us see how to extend *keyAlign* (in Section 6) to implement *relAlign*. We do this extension by two steps. First, we consider an alignment that only deals with filtering of source elements. We extend *keyAlign* with two new arguments, one is the predicate p for filtering source elements, and the other is a function h for hiding/concealing source elements if their corresponding elements are removed from the view. As seen below, *pAlign* has a similar structure to that of *keyAlign*,

where we refine the third case of *keyAlign* into two cases (the third and the fourth cases of *pAlign*).

```
pAlign :: forall s v k ∘ (Show s, Show v, Eq k)
  ⇒ (s → Bool)   { predicate }
    → (s → k) → (v → k) → BiGUL s v → (v → s)
    → (s → Maybe s)   { conceal function }
    → BiGUL [s] [v]
pAlign p ks kv b c h = Case
  [$(normalSV ℙ⟦ [] ⟧ ℙ⟦ [] ⟧ ℙ⟦ [] ⟧)
     ⟹ $(update ℙ⟦ [] ⟧ ℙ⟦ [] ⟧ 𝔻⟦ ⟧)
  ,$(normal ⟦ λ(s : ss) (v : vs) → p s ∧ ks s ≡ kv v ⟧ ⟦ λ(s : ss) → p s ⟧)
     ⟹ $(update ℙ⟦ x : xs ⟧ ℙ⟦ x : xs ⟧ 𝔻⟦ x = b; xs = pAlign p ks kv b c h ⟧)
  ,$(adaptive ⟦ λ(s : ss) v → p s ∧ null v ⟧)
     ⟹ λ(s : ss) v → maybe [] (:[]) (h s) ++ ss
  ,$(normal ⟦ λ(s : ss) v → not (p s) ⟧ ⟦ λ(s : ss) → not (p s) ⟧)
     ⟹ $(update ℙ⟦ _ : xs ⟧ ℙ⟦ xs ⟧ 𝔻⟦ xs = pAlign p ks kv b c h ⟧)
  ,$(adaptive ⟦ λss (v : vs) → kv v ∈ map ks (filter p ss) ⟧)
     ⟹ λss (v: _) → uncurry (:) (extract (kv v) ss)
  ,$(adaptiveSV ℙ⟦ _ ⟧ ℙ⟦ _: _ ⟧)
     ⟹ λss (v: _) → filterCheck p (c v) : ss
  ]
  where
    extract :: k → [s] → (s, [s])
    extract k (x : xs) | p x ∧ ks x ≡ k = (x, xs)
                       | otherwise = let (y, ys) = extract k xs
                                     in (y, x : ys)
    filterCheck p v | p v = v
                    | otherwise = error "error in filter checking"
```

To test, recall the example in Section 6. Consider the following use of *pAlign*:

```
pSelProj = pAlign (λ(k, (n, s)) → s > 1000) fst fst bx cr′ (const Nothing)
  where cr′ (k, n) = (k, (n, 2000))
```

we have:

```
*Brul> get pSelProj employees
Just [(2,"Jeremy")]
*Brul> put pSelProj employees updatedEmployees0
Just [(0,("Zhenjiang",1000)),(1,("Josh",400)),(0,("Zhenjiang",2000)),
(2,("Jeremy",2000))]
```

Second, we extend *pAlign* to deal with functional dependency consistency when updates happen. To this end, we add a new parameter *fd*, a function for updating source records to conform functional dependency. Surprisingly, this

is simple. It is suffice to extend the fourth case of *pAlign* to apply *fd* when inconsistency happens.

```
relAlign :: forall s v k ∘ (Show s, Show v, Eq k, Eq s)
  ⇒ (s → Bool) → (s → k) → (v → k)
    → BiGUL s v → (v → s) → (s → Maybe s)
    → (s → s)   { dependency maintaining function }
    → BiGUL [s] [v]
relAlign p ks kv b c h fd = Case
  [$(normalSV ℙ⟦ [ ] ⟧ ℙ⟦ [ ] ⟧ ℙ⟦ [ ] ⟧)
     ⟹ $(update ℙ⟦ [ ] ⟧ ℙ⟦ [ ] ⟧ 𝔻⟦ ⟧)
  ,$(normal ⟦ λ(s : ss) (v : vs) → p s ∧ ks s ≡ kv v ⟧ ⟦ λ(s : ss) → p s ⟧)
     ⟹ $(update ℙ⟦ x : xs ⟧ ℙ⟦ x : xs ⟧ 𝔻⟦ x = b; xs = relAlign p ks kv b c h fd ⟧)
  ,$(adaptive ⟦ λ(s : ss) v → p s ∧ null v ⟧)
     ⟹ λ(s : ss) v → maybe [ ] ((:[ ]) ∘ filterCheck p ∘ fd) (h s) ++ ss
  ,$(normal ⟦ λ(s : ss) v → not (p s) ⟧ ⟦ λ(s : ss) → not (p s) ⟧)
     ⟹ Case
       [$(adaptive ⟦ λ(s: _) _ → fd s ≢ s ⟧)
          ⟹ λ(s : ss) _ → filterCheck (not ∘ p) (fd s) : ss
       ,$(normal ⟦ \_ _ → True ⟧ ⟦ const True ⟧)
          ⟹ $(update ℙ⟦ _ : xs ⟧ ℙ⟦ xs ⟧ 𝔻⟦ xs = relAlign p ks kv b c h fd ⟧)
       ]
  ,$(adaptive ⟦ λss (v : vs) → kv v ∈ map ks (filter p ss) ⟧)
     ⟹ λss (v: _) → uncurry (:) (extract (kv v) ss)
  ,$(adaptiveSV ℙ⟦ _ ⟧ ℙ⟦ _: _ ⟧)
     ⟹ λss (v: _) → filterCheck p (c v) : ss
  ]
  where
    extract :: k → [s] → (s, [s])
    extract k (x : xs) | p x ∧ ks x ≡ k = (x, xs)
                       | otherwise = let (y, ys) = extract k xs
                                     in (y, x : ys)
    filterCheck p v | p v = v
                    | otherwise = error "error in filter checking"
```

7.3 Describing update policies in selection/projection

With *relAlign*, we can describe various update policies for the selection/projection queries. To be concrete, consider the following selection/projection query:

```
select Track, Rating, Album, Quantity as v
from s
where Quantity > 2
```

Let us see how to write a single BiGUL program so that its *get* does the above query and its *put* describes an intended update policy.

The first BiGUL program is *u0* below.

```
u0 :: RType → (Record → Record) → BiGUL [Record] [Record]
u0 d =
  relAlign
    (λr → (r !! 4) > RInt 2)
    (λs → (s !! 0, s !! 3))
    (λv → (v !! 0, v !! 2))
    $(update ℙ⟦(t : _ : r : a : q : [])⟧
      ℙ⟦(t : r : a : q : [])⟧
      𝔻⟦t = Replace; r = Replace; a = Replace; q = Replace⟧)
    (λ(t : r : a : q : []) → (t : d : r : a : q : []))
    (λrs → Nothing)
```

It matches the source records whose *Quantity* is greater than 2 with the view records by the key (*Track*, *Album*). There are three cases:

- A source record is matched with a view record: we first use a rearrangement function to rearrange the view from a four-element list $[t, r, a, q]$ to a five-element list $[t, \_, r, a, q]$ with the second element marked as underscore. This rearrangement function reshapes the view to match the shape of the source. Then, the element in the source is *Replace*d by the corresponding element in the view.
- A view record that has no matching source record: a new source record is created with a default value d filled into the Date.
- A source record that has no matching view record: we simply delete this record by return *Nothing*.

Now if we would like to hide the source record by setting its Quantity to 0 rather than deleting it if it has no marching view record, we can simply change the last line of *u0* and get *u1* as follows.

```
u1 :: RType → (Record → Record) → BiGUL [Record] [Record]
u1 d =
  relAlign
    (λr → (r !! 4) > RInt 2)
    (λs → (s !! 0, s !! 3))
    (λv → (v !! 0, v !! 2))
    $(update ℙ⟦(t : _ : r : a : q : [])⟧
      ℙ⟦(t : r : a : q : [])⟧
      𝔻⟦t = Replace; r = Replace; a = Replace; q = Replace⟧)
    (λ(t : r : a : q : []) → (t : d : r : a : q : []))
    (λ(t : d : r : a : _ : []) → Just (t : d : r : a : RInt 0 : []))
```

To be concrete, let us see some concrete running examples of using *u0*. Recall *s* defined in Section 7.1. We can confirm that *get* does the query given at the start of this subsection.

```
*Brul> get (u0 (RInt (-1)) id) s
Just
[[RString "Lullaby",RInt 3,RString "Show",RInt 3],
[RString "Lovesong",RInt 5,RString "Disintegration",RInt 4],
[RString "Trust",RInt 4,RString "Wish",RInt 5]]
```

Now suppose that we change the above view to the following:

$$v = [[\mathit{RString}\ \texttt{"Lullaby"}, \mathit{RInt}\ 4, \mathit{RString}\ \texttt{"Show"}, \mathit{RInt}\ 3] \\ , [\mathit{RString}\ \texttt{"Lovesong"}, \mathit{RInt}\ 5, \mathit{RString}\ \texttt{"Disintegration"}, \mathit{RInt}\ 7] \\ ]$$

and we can reflect this change to the source by performing *put* as follows.

```
*Brul> put (u0 (RInt (-1)) (fdFun sfdMap vfdMap svMap v)) s v
Just
[[RString "Lullaby",RInt 1989,RInt 4,RString "Galore",RInt 1],
[RString "Lullaby",RInt 1989,RInt 4,RString "Show",RInt 3],
[RString "Lovesong",RInt 1989,RInt 5,RString "Galore",RInt 1],
[RString "Lovesong",RInt 1989,RInt 5,RString "Disintegration",RInt 7]]
```

Note that the above (*fdFun sfdMap vfdMap svMap v*) denotes a function generated by applying *fdFun* to several dependency mappings and view v. We omit the definition of *fdFun* here.

8 Parsing and reflective printing

When we mention the *front-end* of a compiler, we usually think of a *parser* that turns *concrete syntax*, which is designed to be programmer-friendly and provides convenient syntactic sugar, into *abstract syntax*, which is concise, structured, and easily manipulable by the compiler back-end. There is another direction, though, in which a *printer* turns abstract syntax back into concrete syntax. This is useful, for example, for reporting the result of compiler optimizations done on abstract syntax to the programmer, who knows only concrete syntax. In this case, though, we would want to print the optimized program in a form that is as close to the original program as possible, so the programmer can spot what are changed — and not changed — correctly and more easily. This is where the notion of *reflective printing* comes in: By taking both the original concrete program and the optimized abstract program as input, we can try to retain the look of the original program as much as possible. Below we will use a simplified arithmetic expression language to explain how reflective printing can be implemented in BiGUL.

8.1 Well-behavedness

It is probably obvious that the idea of reflective printing comes from *put* transformations; parsing, then, is the *get* direction. Before we proceed to implement

parsing and reflective printing in BiGUL, a natural question to ask is: Is well-behavedness meaningful in the context of parsing of reflective printing? The answer is yes, especially for PutGet: An abstract syntax tree (AST) may be thought of as a concise and canonical representation of a concrete program, so it would be strange if a concrete program printed from an AST could not be parsed back to the same AST. GetPut, on the other hand, is in fact not strong enough for our purpose, as it only says that, when an AST is unmodified, printing it reflectively to the original program does not change anything, whereas we would have liked to also say that "small" changes to the AST leads to only "small" changes to the concrete program. It is at least a good start to have GetPut, though. We thus conclude that BiGUL is indeed a suitable language for implementing reflective printers and corresponding parsers.

8.2 Additive expressions

Here we use a minimal example which is simple and yet can demonstrate what reflective printing is capable of. Consider the following abstract syntax of arithmetic expressions consisting of integer constant, addition, and subtraction:

```
data Arith = Num Int
           | Add Arith Arith
           | Sub Arith Arith
  deriving Show
```

This is a nice representation for the compiler, but we cannot expect the programmer to write something like "*Sub* (*Num* 1) (*Add* (*Num* 2) (*Num* 3))", and should provide a concrete syntax so that they can write "$1 - (2 + 3)$". Such a concrete syntax is usually defined in terms of a BNF grammar:

```
Exp    → Exp '+' Factor
       | Exp '-' Factor
       | Factor
Factor → Int
       | '-' Factor
       | '(' Exp ')'
```

The two-level structure of *Exp* and *Factor* ensures that plus and minus associate to the left by default; to change association, we should use parentheses. And, to spice up the problem a little, we allow minus to be used also as a negative sign, as specified by the second production rule for *Factor*. BiGUL deals with structured data only, so we should represent a string generated using this grammar as a concrete syntax tree of the following type:

```
data Exp = Plus  Exp Factor
         | Minus Exp Factor
         | EF    Factor
         | ENull
```

```
data Factor = Lit  Int
            | Neg Factor
            | Paren Exp
            | FNull
```

Apart from the *Null* constructors, which are inserted to represent incomplete trees that can occur during reflective printing, these two datatypes are in direct correspondence with the grammar, so it is easy to recover the string from a concrete syntax tree:

```
instance Show Exp where
  show (Plus  e f) = show e ++ "+" ++ show f
  show (Minus e f) = show e ++ "-" ++ show f
  show (EF      f) = show f
  show  ENull    = "."
instance Show Factor where
  show (Lit   n) = show n
  show (Neg   f) = "-" ++ show f
  show (Paren e) = "(" ++ show e ++ ")"
  show  FNull    = "."
```

Conversely, using modern parser technologies like Haskell's `parsec` parser combinator library, we can easily implement a "concrete parser" that turns a string into a concrete syntax tree:

```
parseExp :: String → Either ParseError Exp
unsafeParseExp   :: String → Exp
unsafeParseExp s = let (Right e) = parseExp s in e
```

The rest of the job is then write a BiGUL program between *Exp* and *Arith*.

8.3 Reflective printing in BiGUL

The program is basically a case analysis: For example, when the concrete side is a plus and the abstract side is an addition, they match, and we can go into their sub-trees recursively. For the concrete side, the right sub-tree is of type *Factor* instead of *Exp*, so in fact we will write two (mutually recursive) programs:

```
pExpArith    :: BiGUL Exp Arith
pExpArith    = Case ⊥
pFactorArith :: BiGUL Factor Arith
pFactorArith = Case ⊥
```

The branch for plus and addition can then be written as:

```
$(update ℙ⟦ Plus l r ⟧ ℙ⟦ Add l r ⟧
         𝔻⟦ l = pExpArith; r = pFactorArith ⟧)
```

Following the same line of thought, we can fill in other branches to relate all abstract constructors with concrete production rules:

```
pExpArith :: BiGUL Exp Arith
pExpArith = Case
  [$(normalSV ℙ⟦ Plus _ _ ⟧ ℙ⟦ Add _ _ ⟧ ℙ⟦ Plus _ _ ⟧)
     ⟹ $(update ℙ⟦ Plus l r ⟧ ℙ⟦ Add l r ⟧
                ℙ⟦ l = pExpArith; r = pFactorArith ⟧)
  ,$(normalSV ℙ⟦ Minus _ _ ⟧ ℙ⟦ Sub _ _ ⟧ ℙ⟦ Minus _ _ ⟧)
     ⟹ $(update ℙ⟦ Minus l r ⟧ ℙ⟦ Sub l r ⟧
                ℙ⟦ l = pExpArith; r = pFactorArith ⟧)
  ,$(normalSV ℙ⟦ EF _ ⟧ ℙ⟦ _ ⟧ ℙ⟦ EF _ ⟧)
     ⟹ $(update ℙ⟦ EF t ⟧ ℙ⟦ t ⟧
                ℙ⟦ t = pFactorArith ⟧)
  ]
pFactorArith ::  BiGUL Factor Arith
pFactorArith =   Case
  [$(normalSV ℙ⟦ Lit _ ⟧ ℙ⟦ Num _ ⟧ ℙ⟦ Lit _ ⟧)
     ⟹ $(update ℙ⟦ Lit i ⟧ ℙ⟦ Num i ⟧ ℙ⟦ i = Replace ⟧)
  ,$(normalSV ℙ⟦ Neg _ ⟧ ℙ⟦ Sub (Num 0) _ ⟧ ℙ⟦ Neg _ ⟧)
     ⟹ $(update ℙ⟦ Neg t ⟧ ℙ⟦ Sub (Num 0) t ⟧ ℙ⟦ t = pFactorArith ⟧)
  ,$(normalSV ℙ⟦ Paren _ ⟧ ℙ⟦ _ ⟧ ℙ⟦ Paren _ ⟧)
     ⟹ $(update ℙ⟦ Paren t ⟧ ℙ⟦ t ⟧ ℙ⟦ t = pExpArith ⟧)
  ]
```

This covers only "normal" cases though, namely when the source and view are "the same" except for parentheses and literals. What about the cases when the source and view have mismatched shapes? For these cases, we need adaptation. Corresponding to each branch we have already written, we add an adaptive branch which looks at the shape of the view only, throws away a mismatched source, and creates an incomplete one whose shape matches that of the view; the source will be completely created through recursive processing. For example, corresponding to the plus/addition branch, we write:

```
$(adaptiveSV ℙ⟦ _ ⟧ ℙ⟦ Add _ _ ⟧)
  ⟹ λ_ _ → Plus ENull FNull
```

The full programs are:

```
pExpArith :: BiGUL Exp Arith
pExpArith = Case
  [$(normalSV ℙ⟦ Plus _ _ ⟧ ℙ⟦ Add _ _ ⟧ ℙ⟦ Plus _ _ ⟧)
     ⟹ $(update ℙ⟦ Plus l r ⟧ ℙ⟦ Add l r ⟧
       ℙ⟦ l = pExpArith; r = pFactorArith ⟧)
  ,$(normalSV ℙ⟦ Minus _ _ ⟧ ℙ⟦ Sub _ _ ⟧ ℙ⟦ Minus _ _ ⟧)
```

```
      ⟹ $(update ℙ⟦ Minus l r ⟧ ℙ⟦ Sub l r ⟧
        𝔻⟦ l = pExpArith; r = pFactorArith ⟧)
    , $(normalSV ℙ⟦ EF _ ⟧ ℙ⟦ _ ⟧ ℙ⟦ EF _ ⟧)
      ⟹ $(update ℙ⟦ EF t ⟧ ℙ⟦ t ⟧
        𝔻⟦ t = pFactorArith ⟧)
    , $(adaptiveSV ℙ⟦ _ ⟧ ℙ⟦ Add _ _ ⟧)
      ⟹ λ_ _ → Plus ENull FNull
    , $(adaptiveSV ℙ⟦ _ ⟧ ℙ⟦ Sub _ _ ⟧)
      ⟹ λ_ _ → Minus ENull FNull
    , $(adaptiveSV ℙ⟦ _ ⟧ ℙ⟦ _ ⟧)
      ⟹ λ_ _ → EF FNull
    ]
pFactorArith :: BiGUL Factor Arith
pFactorArith = Case
    [$(normalSV ℙ⟦ Lit _ ⟧ ℙ⟦ Num _ ⟧ ℙ⟦ Lit _ ⟧)
      ⟹ $(update ℙ⟦ Lit i ⟧ ℙ⟦ Num i ⟧ 𝔻⟦ i = Replace ⟧)
    , $(normalSV ℙ⟦ Neg _ ⟧ ℙ⟦ Sub (Num 0) _ ⟧ ℙ⟦ Neg _ ⟧)
      ⟹ $(update ℙ⟦ Neg t ⟧ ℙ⟦ Sub (Num 0) t ⟧ 𝔻⟦ t = pFactorArith ⟧)
    , $(normalSV ℙ⟦ Paren _ ⟧ ℙ⟦ _ ⟧ ℙ⟦ Paren _ ⟧)
      ⟹ $(update ℙ⟦ Paren t ⟧ ℙ⟦ t ⟧ 𝔻⟦ t = pExpArith ⟧)
    , $(adaptiveSV ℙ⟦ _ ⟧ ℙ⟦ Num _ ⟧)
      ⟹ λ_ _ → Lit 0
    , $(adaptiveSV ℙ⟦ _ ⟧ ℙ⟦ Sub (Num 0) _ ⟧)
      ⟹ λ_ _ → Neg FNull
    , $(adaptiveSV ℙ⟦ _ ⟧ ℙ⟦ _ ⟧)
      ⟹ λ_ _ → Paren ENull
    ]
```

8.4 Reflecting optimizations and evaluation sequences

The BiGUL programs, being bidirectional, can be executed in the *put* direction as a reflective printer, or in the *get* direction as a parser. Let us look at parsing first. For example:

```
*Main> get pExpArith (unsafeParseExp "(-(3+4))")
Just (Sub (Num 0) (Add (Num 3) (Num 4)))
```

Note that a unary minus is considered as syntactic sugar, and is desugared into a subtraction whose left operand is zero. Also note that parentheses are turned into correct structure of the abstract syntax tree, and nothing more — excessive parentheses are cleanly discarded.

For reflective printing, as we mentioned, one application is reporting what compiler optimizations do. We can replace the sub-expression 3 + 4 with its

value 1, for example, and the reflective printer will be able to retain the excessive parentheses:

```
*Main> put pExpArith (unsafeParseExp "(-(3+4))")
                     (Sub (Num 0) (Num 7))
Just (-(7))
```

Notice also that the unary minus is preserved. If the original concrete expression uses a binary minus instead, it will be preserved as well:

```
*Main> put pExpArith (unsafeParseExp "(0-(3+4))")
                     (Sub (Num 0) (Num 7))
Just (0-(7))
```

More generally, the steps in an evaluation sequence of an abstract syntax tree can all be reflected to concrete syntax. For example, starting from:

```
*Main> get pExpArith (unsafeParseExp "1+(2+3)")
Just (Add (Num 1) (Add (Num 2) (Num 3)))
```

it takes two steps to evaluate this expression:

```
*Main> put pExpArith (unsafeParseExp "1+(2+3)")
                     (Add (Num 1) (Num 5))
Just 1+(5)
*Main> put pExpArith (unsafeParseExp "1+(5)") (Num 6)
Just 6
```

This means that if we have an evaluator on the abstract syntax, we will automatically get an evaluator on the concrete syntax!

A reflective printer can also be used as an ordinary printer by setting the original source to an empty one. For example:

```
*Main> put pExpArith ENull (Sub (Num 0) (Add (Num 1) (Num 1)))
Just 0-(1+1)
```

You have probably noticed that the subtraction is reflected as a binary minus instead of a unary one, despite that the left operand is zero. This behavior is easily customizable: By adding an adaptive branch before the one dealing generically with *Sub* in *pExpArith*:

$$^{\$}(adaptiveSV\ ^{\mathbb{P}}[\![\ \_\ ]\!]\ ^{\mathbb{P}}[\![\ Sub\ (Num\ 0)\ \_\ ]\!])$$
$$\Longrightarrow \lambda\_\ \_ \to EF\ FNull$$

the above abstract syntax tree can be printed as:

```
*Main> put pExpArith ENull (Sub (Num 0) (Add (Num 1) (Num 1)))
Just -(1+1)
```

8.5 A domain-specific language

As a final remark, the above programs may look long, but at the core of them are merely the correspondences between concrete production rules and abstract constructors. We can design a domain-specific language (DSL) that expresses such correspondences concisely, and then expand programs in this DSL to BiGUL. In fact, we have already done so, and the DSL is called *BiYacc*. For example, all the programs we have written can be generated from the following eight-line BiYacc program:

```
Arith +> Exp
Add l r +> (l +> Exp) '+' (r +> Factor);
Sub l r +> (l +> Exp) '-' (r +> Factor);
f       +> (f +> Factor);

Arith +> Factor
Num n         +> (n +> Int);
Sub (Num 0) r +> '-' (r +> Factor);
f             +> '(' (f +> Exp) ')';
```

See our SLE 2016 paper [17] for more interesting experiments about reflective printing, done on a more realistic imperative language.

References

1. Bohannon, A., Foster, J.N., Pierce, B.C., Pilkiewicz, A., Schmitt, A.: Boomerang: resourceful lenses for string data. In: POPL 2008. pp. 407–419. ACM (2008)
2. Bohannon, A., Pierce, B.C., Vaughan, J.A.: Relational lenses: a language for updatable views. In: PODS 2006. pp. 338–347. ACM (2006)
3. Bohannon, A., Pierce, B.C., Vaughan, J.A.: Relational lenses: A language for updatable views. In: Principles of Database Systems. pp. 338–347. PODS '06, ACM (2006), http://doi.acm.org/10.1145/1142351.1142399
4. Fischer, S., Hu, Z., Pacheco, H.: A clear picture of lens laws - functional pearl. In: Hinze, R., Voigtl"ander, J. (eds.) Mathematics of Program Construction - 12th International Conference, MPC 2015, K"onigswinter, Germany, June 29 - July 1, 2015. Proceedings. Lecture Notes in Computer Science, vol. 9129, pp. 215–223. Springer (2015), http://dx.doi.org/10.1007/978-3-319-19797-5
5. Fischer, S., Hu, Z., Pacheco, H.: The essence of bidirectional programming. SCIENCE CHINA Information Sciences 58(5), 1–21 (2015)
6. Foster, J.: Bidirectional Programming Languages. Ph.D. thesis, University of Pennsylvania (December 2009)
7. Foster, J.N., Greenwald, M.B., Moore, J.T., Pierce, B.C., Schmitt, A.: Combinators for bidirectional tree transformations: A linguistic approach to the view-update problem. ACM Transactions on Programming Languages and Systems 29(3), 17 (2007)
8. Hidaka, S., Hu, Z., Inaba, K., Kato, H., Matsuda, K., Nakano, K.: Bidirectionalizing graph transformations. In: ICFP 2010. pp. 205–216. ACM (2010)
9. Hofmann, M., Pierce, B.C., Wagner, D.: Symmetric lenses. In: POPL 2011. pp. 371–384. ACM (2011)

10. Ko, H., Zan, T., Hu, Z.: Bigul: a formally verified core language for putback-based bidirectional programming. In: Erwig, M., Rompf, T. (eds.) Proceedings of the 2016 ACM SIGPLAN Workshop on Partial Evaluation and Program Manipulation, PEPM 2016, St. Petersburg, FL, USA, January 20 - 22, 2016. pp. 61–72. ACM (2016), http://doi.acm.org/10.1145/2847538.2847544
11. Matsuda, K., Hu, Z., Nakano, K., Hamana, M., Takeichi, M.: Bidirectionalization transformation based on automatic derivation of view complement functions. In: ICFP 2007. pp. 47–58. ACM (2007)
12. Pacheco, H., Hu, Z., Fischer, S.: Monadic combinators for "putback" style bidirectional programming. In: PEPM '14. pp. 39–50. ACM (2014)
13. Pacheco, H., Zan, T., Hu, Z.: Biflux: A bidirectional functional update language for XML. In: Chitil, O., King, A., Danvy, O. (eds.) Proceedings of the 16th International Symposium on Principles and Practice of Declarative Programming, Kent, Canterbury, United Kingdom, September 8-10, 2014. pp. 147–158. ACM (2014), http://doi.acm.org/10.1145/2643135.2643141
14. Voigtländer, J.: Bidirectionalization for free! (pearl). In: POPL 2009. pp. 165–176. ACM (2009)
15. Xiong, Y., Liu, D., Hu, Z., Zhao, H., Takeichi, M., Mei, H.: Towards automatic model synchronization from model transformations. In: ASE 2007. pp. 164–173. ACM (2007)
16. Zan, T., Liu, L., Ko, H., Hu, Z.: Brul: A putback-based bidirectional transformation library for updatable views. In: Anjorin, A., Gibbons, J. (eds.) Proceedings of the 5th International Workshop on Bidirectional Transformations, Bx 2016, co-located with The European Joint Conferences on Theory and Practice of Software, ETAPS 2016, Eindhoven, The Netherlands, April 8, 2016. CEUR Workshop Proceedings, vol. 1571, pp. 77–89. CEUR-WS.org (2016), http://ceur-ws.org/Vol-1571/paper_3.pdf
17. Zhu, Z., Zhang, Y., Ko, H.S., Martins, P., Saraiva, J., Hu, Z.: Parsing and reflective printing, bidirectionally. In: Software Language Engineering. pp. 2–14. SLE'16, ACM (2016)